

Guia de estudio Java FullstackCodigo Tecsups

Spring Boot, microservicios, APIs externas, testing y React explicados desde tus propios proyectos.

Repositorio analizado: JAVA-FULLSTACK-CODIGO-TECSUP. Inventario: 15 proyectos, 275 clases Java y 15 archivos JavaScript/React.

La idea de esta guia es que puedas abrir el codigo y leerlo con mapa mental: que rol cumple cada clase, que revisar primero y como conectarlo con el roadmap.

Analogias base

- Spring Boot es una ciudad con servicios publicos: configura infraestructura para que te enfoques en el negocio.
- Controller es ventanilla: recibe solicitudes HTTP.
- Service es cocina/oficina interna: aplica reglas.
- Repository es almacen: guarda y trae datos.
- Entity es ficha oficial: representa el dato persistente.
- DTO es formulario: transporta solo lo necesario.
- Config es tablero electrico: conecta beans, filtros y clientes.
- Test es checklist: confirma que una pieza se comporta bien.

Roadmap Spring Boot: cobertura

Tema	Estado	Evidencia en tus proyectos
Terminology	Visto fuerte	Se usan conceptos Spring Boot, bean, controller, service, repository, DTO, entity, microservice y cliente HTTP en casi todos los proyectos.
Architecture	Visto fuerte	Capas classicas en APIs REST y arquitectura hexagonal en los proyectos RENIEC/SUNAT.
Why use Spring?	Visto medio	Se evidencia por uso practico: autoconfiguracion, inyeccion, starters y productividad; falta una comparacion teorica explicita.
Configuration	Visto fuerte	application.properties, bootstrap.properties, Config Server, Vault y variables de entorno.
Dependency Injection	Visto fuerte	@Service, @Repository, @Configuration, constructores Lombok y beans inyectados.
Spring IoC	Visto fuerte	El contenedor crea controladores, servicios, repositorios, filtros, configs y clientes.
Spring AOP	Pendiente / indirecto	No hay aspectos propios. Se ve indirectamente en seguridad, proxies JPA y anotaciones Spring.
Spring MVC	Visto fuerte	Controladores REST, RequestMapping, GetMapping, PostMapping, RequestBody, PathVariable y ResponseEntity.
Annotations	Visto fuerte	Spring, JPA, Lombok, Swagger/OpenAPI, Mockito, JUnit y Security.
Spring Bean Scope	Pendiente	No se observan scopes personalizados como request, session o prototype.
Authentication	Visto fuerte	ms-auth implementa login, validacion de tokens y usuarios.
Authorization	Visto fuerte	Roles ADMIN/USER y reglas de SecurityFilterChain.
OAuth2	Pendiente	No se observa flujo OAuth2/OIDC.
JWT Authentication	Visto fuerte	JwtService, JwtAuthenticationFilter, refresh token y validacion.
Transactions	Visto medio	Se usa JPA y repositorios; faltan usos explicitos de @Transactional en reglas criticas.
Relationships	Visto fuerte	Entidades relacionadas en aerolinea y gestion academica.
Entity Lifecycle	Visto medio	Se usan entidades JPA; no se ven callbacks como @PrePersist o @PostLoad.
Hibernate	Visto fuerte	JPA/Hibernate sostiene entidades, repositorios y relaciones.
Spring Data JPA	Visto fuerte	Repositories JpaRepository en casi todas las APIs persistentes.
Spring Data MongoDB	Visto inicial	Aparece repositorio Mongo en el proyecto hexagonal RENIEC.
Spring Data JDBC	Pendiente	No se observa uso directo de Spring Data JDBC.
Spring Boot Starters	Visto fuerte	POMs usan starters web, data-jpa, security, test, validation, openfeign, cloud, etc.
Autoconfiguration	Visto fuerte	Se aprovecha en servidor embebido, MVC, JPA, security y tests.
Actuators	Pendiente	No se observa Actuator como dependencia o endpoints de monitoreo.
Embedded Server	Visto fuerte	Todos los proyectos Spring Boot arrancan como apps con servidor embebido.
Microservices	Visto fuerte	ms-auth, ms-productos, ms-ordenes y ms-config-repository forman ecosistema.
Spring Cloud Gateway	Pendiente	No hay gateway dedicado.
Cloud Config	Visto fuerte	ms-config-repository y bootstrap.properties.
Spring Cloud Circuit Breaker	Pendiente	No hay Resilience4j/CircuitBreaker.
Spring Cloud OpenFeign	Visto fuerte	OpenFeign se usa en proyectos RENIEC/SUNAT.
Micrometer	Pendiente	No se observa instrumentacion directa.
Eureka	Pendiente	No se observa service discovery.
Servlet	Visto indirecto	Spring MVC y filtros de seguridad corren sobre la base servlet.
JSP Files	Pendiente	No hay vistas JSP; los proyectos son REST y React.
Spring MVC Architecture	Visto fuerte	Controller -> Service -> Repository y DTOs.
Spring MVC Components	Visto fuerte	Controllers, handlers de excepcion, DTOs, requests, responses y configs.
JPA Test	Visto inicial	Hay pruebas, pero no se observa un enfoque principal con @DataJpaTest.
Mock MVC	Visto fuerte	Tests de controladores con MockMvc en microservicios y unit-testing.

Tema	Estado	Evidencia en tus proyectos
@SpringBootTest	Visto fuerte	Clases de test de contexto en varios proyectos.
@MockBean	Visto medio	Se practican mocks con Mockito; revisar @MockBean como siguiente paso si se quiere aislar contexto Spring.

Secuencia recomendada de repaso

1. APIs CRUD clásicas: aerolínea y gestión académica.
2. Consumo de APIs externas: OpenFeign, RestTemplate, Retrofit y WebClient.
3. Arquitectura hexagonal: RENIEC y SUNAT.
4. Patrones de diseño: Strategy, Adapter y Builder.
5. Testing: unit-testing-empresa.
6. Microservicios: config, auth, productos y ordenes.
7. Frontend final: React.

Aerolinea API Spring Boot

Tema principal: API REST, JPA, MySQL y relaciones entre aerolineas, aviones, pilotos, pasajeros, vuelos y boletos.

Analogia del proyecto: Es como administrar un aeropuerto: cada entidad es una oficina, los servicios coordinan operaciones y los controladores atienden ventanillas.

Dependencias y piezas visibles

Modulo Maven: spring-boot-starter-parent, lombok, spring-boot-starter-data-jpa, spring-boot-starter-web

Flujo mental

HTTP -> Controller -> Service -> Repository/Ciente externo -> Entity/DTO -> Response.

Clases y archivos explicados

Application

Archivo: aerolinea-api-spring-boot/src/main/java/com/codigo/spring/Application.java

Tipo: class. Rol: Punto de arranque. Paquete: com.codigo.spring.

- Enciende Spring Boot, escanea componentes y levanta el servidor embebido.
- Metodos para revisar: void main(String[] args).
- Anotaciones importantes: @SpringBootApplication.
- Analogia: Como girar la llave de contacto de un auto.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

OpenApiConfig

Archivo: aerolinea-api-spring-boot/src/main/java/com/codigo/spring/config/OpenApiConfig.java

Tipo: class. Rol: Configuracion. Paquete: com.codigo.spring.config.

- Declara beans, filtros, clientes o ajustes para que Spring los administre.
- Metodos para revisar: OpenAPI aerolineaOpenApi().
- Anotaciones importantes: @Bean, @Configuration.
- Analogia: Como el tablero de interruptores que conecta piezas del sistema.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

AerolineaController

Archivo: aerolinea-api-spring-boot/src/main/java/com/codigo/spring/controller/AerolineaController.java

Tipo: class. Rol: Controlador REST. Paquete: com.codigo.spring.controller.

- Recibe solicitudes HTTP, toma parametros/body y delega al servicio.
- Expone rutas HTTP como @RequestMapping("/api/v1/aerolinea"), @PostMapping("/save"), @GetMapping("/find/all"), @GetMapping("/find/{id}").
- Sus datos/dependencias clave son: final Aerolineaservice aerolineaservice.
- Metodos para revisar: Aerolineaservice save(@Valid @RequestBody Aerolineaservice aerolineaservice), List<Aerolineaservice> findAll(), ResponseBase<Aerolineaservice> findById(@Parameter(description = "ID de la aerolinea", example = "1"), ResponseBase<Aerolineaservice> update(@Parameter(description = "ID de la aerolinea", example = "1"), Aerolineaservice aerolineaservice).
- Anotaciones importantes: @ApiResponse, @ApiResponses, @GetMapping, @Operation, @Parameter, @PathVariable, @PostMapping, @PutMapping.
- Analogia: Como una ventanilla: escucha al usuario y pasa el pedido a la oficina correcta.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

AvionController

Archivo: aerolinea-api-spring-boot/src/main/java/com/codigo/spring/controller/AvionController.java

Tipo: class. Rol: Controlador REST. Paquete: com.codigo.spring.controller.

- Recibe solicitudes HTTP, toma parametros/body y delega al servicio.
- Expone rutas HTTP como `@RequestMapping("/api/v1/avion")`, `@PostMapping("/save")`, `@GetMapping("/find/{id}")`, `@GetMapping("/find/modelo")`.
- Sus datos/dependencias clave son: `final AvionService avionService`.
- Metodos para revisar: `AvionEntity save(@Valid @RequestBody AvionEntity avionEntity)`, `ResponseBase<AvionResponse> findById(@Parameter(description = "ID del avion", example = "1"), List<AvionResponse> findAll(@Parameter(description = "Modelo del avion", example = "Airbus A320"), List<AvionResponse> findAllCapacidad(@Parameter(description = "Capacidad minima", example = "100"), ResponseBase<AvionResponse> updateAerolinea(@Parameter(description = "ID del avion", example = "1"), AvionController(AvionService avionService))`.
- Anotaciones importantes: `@ApiResponse`, `@ApiResponses`, `@GetMapping`, `@Operation`, `@Parameter`, `@PathVariable`, `@PostMapping`, `@PutMapping`.
- Analogia: Como una ventanilla: escucha al usuario y pasa el pedido a la oficina correcta.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

BoletoController

Archivo: `aerolinea-api-spring-boot/src/main/java/com/codigo/spring/controller/BoletoController.java`

Tipo: class. Rol: Controlador REST. Paquete: com.codigo.spring.controller.

- Recibe solicitudes HTTP, toma parametros/body y delega al servicio.
- Expone rutas HTTP como `@RequestMapping("/api/v1/boleto")`, `@PostMapping("/save")`.
- Sus datos/dependencias clave son: `final BoletoService boletoService`.
- Metodos para revisar: `ResponseBase<BoletoResponse> save(@Valid @RequestBody BoletoRequest boletoRequest)`, `BoletoController(BoletoService boletoService)`.
- Anotaciones importantes: `@ApiResponse`, `@ApiResponses`, `@Operation`, `@PostMapping`, `@RequestBody`, `@RequestMapping`, `@RestController`, `@Tag`.
- Analogia: Como una ventanilla: escucha al usuario y pasa el pedido a la oficina correcta.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

PasajeroController

Archivo: `aerolinea-api-spring-boot/src/main/java/com/codigo/spring/controller/PasajeroController.java`

Tipo: class. Rol: Controlador REST. Paquete: com.codigo.spring.controller.

- Recibe solicitudes HTTP, toma parametros/body y delega al servicio.
- Expone rutas HTTP como `@RequestMapping("/api/v1/pasajero")`, `@PostMapping("/save")`, `@GetMapping("/find/{id}")`, `@GetMapping("/find/boleto/{id}")`.
- Sus datos/dependencias clave son: `final PasajeroService pasajeroService`.
- Metodos para revisar: `ResponseBase<PasajeroResponse> save(@Valid @RequestBody PasajeroEntity pasajeroEntity)`, `ResponseBase<PasajeroResponse> findById(@Parameter(description = "ID del pasajero", example = "1"), ResponseBase<PasajeroInfoResponse> findBoleto(@Parameter(description = "ID del pasajero", example = "1"), PasajeroController(PasajeroService pasajeroService))`.
- Anotaciones importantes: `@ApiResponse`, `@ApiResponses`, `@Content`, `@ExampleObject`, `@GetMapping`, `@Operation`, `@Parameter`, `@PathVariable`.
- Analogia: Como una ventanilla: escucha al usuario y pasa el pedido a la oficina correcta.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

PilotoController

Archivo: `aerolinea-api-spring-boot/src/main/java/com/codigo/spring/controller/PilotoController.java`

Tipo: class. Rol: Controlador REST. Paquete: com.codigo.spring.controller.

- Recibe solicitudes HTTP, toma parametros/body y delega al servicio.
- Expone rutas HTTP como `@RequestMapping("/api/v1/piloto")`, `@PostMapping("/save")`, `@GetMapping("/find/{id}")`, `@GetMapping("/find/all")`.

- Sus datos/dependencias clave son: final `PilotoService pilotoService`.
- Metodos para revisar: `ResponseBase<PilotoResponse> save(@Valid @RequestBody PilotoEntity pilotoEntity)`, `ResponseBase<PilotoResponse> findById(@Parameter(description = "ID del piloto", example = "1"), List<PilotoResponse> findAll(), PilotoController(PilotoService pilotoService))`.
- Anotaciones importantes: `@ApiResponse`, `@ApiResponses`, `@GetMapping`, `@Operation`, `@Parameter`, `@PathVariable`, `@PostMapping`, `@RequestBody`.
- Analogia: Como una ventanilla: escucha al usuario y pasa el pedido a la oficina correcta.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

VueloController

Archivo: `aerolinea-api-spring-boot/src/main/java/com/codigo/spring/controller/VueloController.java`

Tipo: class. Rol: Controlador REST. Paquete: `com.codigo.spring.controller`.

- Recibe solicitudes HTTP, toma parametros/body y delega al servicio.
- Expone rutas HTTP como `@RequestMapping("/api/v1/vuelo")`, `@PostMapping("/save")`, `@GetMapping("/find/{id}")`, `@GetMapping("/find")`.
- Sus datos/dependencias clave son: final `VueloService vueloService`.
- Metodos para revisar: `ResponseBase<VueloResponse> save(@Valid @RequestBody VueloRequest vueloRequest)`, `ResponseBase<VueloResponse> findById(@Parameter(description = "ID del vuelo", example = "1"), List<VueloResponse> findById(@Parameter(description = "Fecha de salida en formato yyyy-MM-dd", example = "2026-05-04"), ResponseBase<VueloResponse> updatePilotos(@Valid @RequestBody VueloRequestUpdatePilotos vueloRequest), VueloController(VueloService vueloService))`.
- Anotaciones importantes: `@ApiResponse`, `@ApiResponses`, `@DateTimeFormat`, `@GetMapping`, `@Operation`, `@Parameter`, `@PathVariable`, `@PostMapping`.
- Analogia: Como una ventanilla: escucha al usuario y pasa el pedido a la oficina correcta.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

AerolineaEntity

Archivo: `aerolinea-api-spring-boot/src/main/java/com/codigo/spring/entity/AerolineaEntity.java`

Tipo: class. Rol: Entidad / modelo persistente. Paquete: `com.codigo.spring.entity`.

- Representa datos de dominio y normalmente se mapea a tablas o documentos.
- Sus datos/dependencias clave son: Integer id, String nombre.
- Anotaciones importantes: `@AllArgsConstructor`, `@Column`, `@Entity`, `@GeneratedValue`, `@Getter`, `@Id`, `@NoArgsConstructor`, `@NotBlank`.
- Analogia: Como una ficha oficial con campos que describen algo del negocio.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

AvionEntity

Archivo: `aerolinea-api-spring-boot/src/main/java/com/codigo/spring/entity/AvionEntity.java`

Tipo: class. Rol: Entidad / modelo persistente. Paquete: `com.codigo.spring.entity`.

- Representa datos de dominio y normalmente se mapea a tablas o documentos.
- Sus datos/dependencias clave son: Integer id, int capacidad, int peso, String modelo, AerolineaEntity aerolinea.
- Anotaciones importantes: `@Column`, `@Entity`, `@GeneratedValue`, `@Getter`, `@Id`, `@JoinColumn`, `@JsonIgnore`, `@ManyToOne`.
- Analogia: Como una ficha oficial con campos que describen algo del negocio.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

BoletosEntity

Archivo: `aerolinea-api-spring-boot/src/main/java/com/codigo/spring/entity/BoletosEntity.java`

Tipo: class. Rol: Entidad / modelo persistente. Paquete: `com.codigo.spring.entity`.

- Representa datos de dominio y normalmente se mapea a tablas o documentos.

- Sus datos/dependencias clave son: int id, int asiento, PasajeroEntity pasajero, VueloEntity vuelo.
- Anotaciones importantes: @Column, @Entity, @GeneratedValue, @Getter, @Id, @JoinColumn, @ManyToOne, @Setter.
- Analogia: Como una ficha oficial con campos que describen algo del negocio.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

PasajeroEntity

Archivo: aerolinea-api-spring-boot/src/main/java/com/codigo/spring/entity/PasajeroEntity.java

Tipo: class. Rol: Entidad / modelo persistente. Paquete: com.codigo.spring.entity.

- Representa datos de dominio y normalmente se mapea a tablas o documentos.
- Sus datos/dependencias clave son: Integer id, String nombre, String apellido.
- Anotaciones importantes: @AllArgsConstructor, @Column, @Entity, @GeneratedValue, @Getter, @Id, @NoArgsConstructor, @NotBlank.
- Analogia: Como una ficha oficial con campos que describen algo del negocio.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

PilotoEntity

Archivo: aerolinea-api-spring-boot/src/main/java/com/codigo/spring/entity/PilotoEntity.java

Tipo: class. Rol: Entidad / modelo persistente. Paquete: com.codigo.spring.entity.

- Representa datos de dominio y normalmente se mapea a tablas o documentos.
- Sus datos/dependencias clave son: int id, String nombre, String apellido, Boolean estado, LocalDate fechaCreacion, LocalDate fechaModificacion.
- Anotaciones importantes: @Column, @Entity, @GeneratedValue, @Getter, @Id, @JoinColumn, @JoinTable, @ManyToMany.
- Analogia: Como una ficha oficial con campos que describen algo del negocio.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

VueloEntity

Archivo: aerolinea-api-spring-boot/src/main/java/com/codigo/spring/entity/VueloEntity.java

Tipo: class. Rol: Entidad / modelo persistente. Paquete: com.codigo.spring.entity.

- Representa datos de dominio y normalmente se mapea a tablas o documentos.
- Sus datos/dependencias clave son: Long id, LocalDate fechaSalida, LocalDate fechaLlegada, String origen, String destino, AvionEntity avion.
- Anotaciones importantes: @Column, @Entity, @GeneratedValue, @Getter, @Id, @JoinColumn, @ManyToMany, @ManyToOne.
- Analogia: Como una ficha oficial con campos que describen algo del negocio.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

GlobalExceptionHandler

Archivo: aerolinea-api-spring-boot/src/main/java/com/codigo/spring/exception/GlobalExceptionHandler.java

Tipo: class. Rol: Excepcion / handler. Paquete: com.codigo.spring.exception.

- Representa errores o traduce excepciones en respuestas HTTP comprensibles.
- Metodos para revisar: ResponseEntity<ResponseBase<List<String>>> handleValidation(MethodArgumentNotValidException exception), ResponseEntity<ResponseBase<List<String>>> handleUnreadableRequest().
- Anotaciones importantes: @ExceptionHandler, @RestControllerAdvice.
- Analogia: Como un protocolo de emergencia que evita respuestas desordenadas.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

AvionMapper

Archivo: aerolinea-api-spring-boot/src/main/java/com/codigo/spring/mapper/AvionMapper.java

Tipo: interface. Rol: Mapper. Paquete: com.codigo.spring.mapper.

- Convierte entidades, requests y responses para separar modelos internos y externos.
- Anotaciones importantes: @Mapper, @Mapping, @Mappings.
- Analogia: Como un traductor entre el idioma de la base de datos y el idioma de la API.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

BoletoMapper

Archivo: aerolinea-api-spring-boot/src/main/java/com/codigo/spring/mapper/BoletoMapper.java

Tipo: interface. Rol: Mapper. Paquete: com.codigo.spring.mapper.

- Convierte entidades, requests y responses para separar modelos internos y externos.
- Anotaciones importantes: @Mapper, @Mapping, @Mappings.
- Analogia: Como un traductor entre el idioma de la base de datos y el idioma de la API.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

PasajeroMapper

Archivo: aerolinea-api-spring-boot/src/main/java/com/codigo/spring/mapper/PasajeroMapper.java

Tipo: interface. Rol: Mapper. Paquete: com.codigo.spring.mapper.

- Convierte entidades, requests y responses para separar modelos internos y externos.
- Anotaciones importantes: @Mapper.
- Analogia: Como un traductor entre el idioma de la base de datos y el idioma de la API.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

PilotoMapper

Archivo: aerolinea-api-spring-boot/src/main/java/com/codigo/spring/mapper/PilotoMapper.java

Tipo: interface. Rol: Mapper. Paquete: com.codigo.spring.mapper.

- Convierte entidades, requests y responses para separar modelos internos y externos.
- Anotaciones importantes: @Mapper.
- Analogia: Como un traductor entre el idioma de la base de datos y el idioma de la API.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

VueloMapper

Archivo: aerolinea-api-spring-boot/src/main/java/com/codigo/spring/mapper/VueloMapper.java

Tipo: interface. Rol: Mapper. Paquete: com.codigo.spring.mapper.

- Convierte entidades, requests y responses para separar modelos internos y externos.
- Anotaciones importantes: @Mapper, @Mapping, @Mappings.
- Analogia: Como un traductor entre el idioma de la base de datos y el idioma de la API.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

AerolineaRepository

Archivo: aerolinea-api-spring-boot/src/main/java/com/codigo/spring/repository/AerolineaRepository.java

Tipo: interface. Rol: Repositorio de datos. Paquete: com.codigo.spring.repository.

- Abstrae la persistencia y permite consultar o guardar entidades.
- Analogia: Como el archivo o almacen donde se guarda informacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

AvionRepository

Archivo: aerolinea-api-spring-boot/src/main/java/com/codigo/spring/repository/AvionRepository.java

Tipo: interface. Rol: Repositorio de datos. Paquete: com.codigo.spring.repository.

- Abstrae la persistencia y permite consultar o guardar entidades.
- Analogia: Como el archivo o almacen donde se guarda informacion.

- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

BoletoRepository

Archivo: `aerolinea-api-spring-boot/src/main/java/com/codigo/spring/repository/BoletoRepository.java`

Tipo: interface. Rol: Repositorio de datos. Paquete: `com.codigo.spring.repository`.

- Abstrae la persistencia y permite consultar o guardar entidades.
- Anotaciones importantes: `@Query`.
- Analogia: Como el archivo o almacen donde se guarda informacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

PasajeroRepository

Archivo: `aerolinea-api-spring-boot/src/main/java/com/codigo/spring/repository/PasajeroRepository.java`

Tipo: interface. Rol: Repositorio de datos. Paquete: `com.codigo.spring.repository`.

- Abstrae la persistencia y permite consultar o guardar entidades.
- Analogia: Como el archivo o almacen donde se guarda informacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

PilotoRepository

Archivo: `aerolinea-api-spring-boot/src/main/java/com/codigo/spring/repository/PilotoRepository.java`

Tipo: interface. Rol: Repositorio de datos. Paquete: `com.codigo.spring.repository`.

- Abstrae la persistencia y permite consultar o guardar entidades.
- Anotaciones importantes: `@Repository`.
- Analogia: Como el archivo o almacen donde se guarda informacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

VueloRepository

Archivo: `aerolinea-api-spring-boot/src/main/java/com/codigo/spring/repository/VueloRepository.java`

Tipo: interface. Rol: Repositorio de datos. Paquete: `com.codigo.spring.repository`.

- Abstrae la persistencia y permite consultar o guardar entidades.
- Anotaciones importantes: `@Query`.
- Analogia: Como el archivo o almacen donde se guarda informacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

BoletoRequest

Archivo: `aerolinea-api-spring-boot/src/main/java/com/codigo/spring/request/BoletoRequest.java`

Tipo: class. Rol: DTO / request / response. Paquete: `com.codigo.spring.request`.

- Transporta datos entre capas o hacia/desde HTTP sin exponer directamente la entidad.
- Sus datos/dependencias clave son: Long idVuelo, int idPasajero, int asiento.
- Anotaciones importantes: `@Getter`, `@Min`, `@NotNull`, `@Setter`.
- Analogia: Como un formulario: solo lleva los datos necesarios para una operacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

VueloRequest

Archivo: `aerolinea-api-spring-boot/src/main/java/com/codigo/spring/request/VueloRequest.java`

Tipo: class. Rol: DTO / request / response. Paquete: `com.codigo.spring.request`.

- Transporta datos entre capas o hacia/desde HTTP sin exponer directamente la entidad.
- Sus datos/dependencias clave son: LocalDate fechaSalida, LocalDate fechaLlegada, String origen, String destino, int avionId.
- Anotaciones importantes: `@Getter`, `@Min`, `@NotBlank`, `@NotNull`, `@Setter`.

- Analogia: Como un formulario: solo lleva los datos necesarios para una operacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

VueloRequestUpdatePilotos

Archivo: aerolinea-api-spring-boot/src/main/java/com/codigo/spring/request/VueloRequestUpdatePilotos.java

Tipo: class. Rol: DTO / request / response. Paquete: com.codigo.spring.request.

- Transporta datos entre capas o hacia/desde HTTP sin exponer directamente la entidad.
- Sus datos/dependencias clave son: Long idVuelo, List<Integer> idsPilotos.
- Anotaciones importantes: @Getter, @NotEmpty, @NotNull, @Setter.
- Analogia: Como un formulario: solo lleva los datos necesarios para una operacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

AvionResponse

Archivo: aerolinea-api-spring-boot/src/main/java/com/codigo/spring/response/AvionResponse.java

Tipo: class. Rol: DTO / request / response. Paquete: com.codigo.spring.response.

- Transporta datos entre capas o hacia/desde HTTP sin exponer directamente la entidad.
- Sus datos/dependencias clave son: int peso, String aerolinea.
- Metodos para revisar: AvionResponse(int capacidad, String modelo, int peso, String aerolinea).
- Anotaciones importantes: @Getter, @Setter.
- Analogia: Como un formulario: solo lleva los datos necesarios para una operacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

AvionResponseBase

Archivo: aerolinea-api-spring-boot/src/main/java/com/codigo/spring/response/AvionResponseBase.java

Tipo: class. Rol: DTO / request / response. Paquete: com.codigo.spring.response.

- Transporta datos entre capas o hacia/desde HTTP sin exponer directamente la entidad.
- Sus datos/dependencias clave son: int capacidad, String modelo.
- Anotaciones importantes: @AllArgsConstructor, @Getter, @NoArgsConstructor, @Setter.
- Analogia: Como un formulario: solo lleva los datos necesarios para una operacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

BoletoResponse

Archivo: aerolinea-api-spring-boot/src/main/java/com/codigo/spring/response/BoletoResponse.java

Tipo: class. Rol: DTO / request / response. Paquete: com.codigo.spring.response.

- Transporta datos entre capas o hacia/desde HTTP sin exponer directamente la entidad.
- Sus datos/dependencias clave son: String nombre, String apellido, int asiento, String origen, String destino, LocalDate fechaSalida.
- Anotaciones importantes: @AllArgsConstructor, @Getter, @Setter.
- Analogia: Como un formulario: solo lleva los datos necesarios para una operacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

PasajeroInfoResponse

Archivo: aerolinea-api-spring-boot/src/main/java/com/codigo/spring/response/PasajeroInfoResponse.java

Tipo: class. Rol: DTO / request / response. Paquete: com.codigo.spring.response.

- Transporta datos entre capas o hacia/desde HTTP sin exponer directamente la entidad.
- Sus datos/dependencias clave son: String nombre, String apellido, String origen, String destino, int asiento, List<BoletoInfo> boletos.
- Anotaciones importantes: @AllArgsConstructor, @Getter, @NoArgsConstructor, @Setter.

- Analogia: Como un formulario: solo lleva los datos necesarios para una operacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

PasajeroResponse

Archivo: aerolinea-api-spring-boot/src/main/java/com/codigo/spring/response/PasajeroResponse.java

Tipo: class. Rol: DTO / request / response. Paquete: com.codigo.spring.response.

- Transporta datos entre capas o hacia/desde HTTP sin exponer directamente la entidad.
- Sus datos/dependencias clave son: Integer id, String nombre, String apellido.
- Anotaciones importantes: @AllArgsConstructor, @Getter, @NoArgsConstructor, @Setter.
- Analogia: Como un formulario: solo lleva los datos necesarios para una operacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

PilotoResponse

Archivo: aerolinea-api-spring-boot/src/main/java/com/codigo/spring/response/PilotoResponse.java

Tipo: class. Rol: DTO / request / response. Paquete: com.codigo.spring.response.

- Transporta datos entre capas o hacia/desde HTTP sin exponer directamente la entidad.
- Sus datos/dependencias clave son: int id, String nombre, String apellido, Boolean estado, LocalDate fechaCreacion, LocalDate fechaModificacion.
- Anotaciones importantes: @AllArgsConstructor, @Getter, @NoArgsConstructor, @Setter.
- Analogia: Como un formulario: solo lleva los datos necesarios para una operacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

ResponseBase

Archivo: aerolinea-api-spring-boot/src/main/java/com/codigo/spring/response/ResponseBase.java

Tipo: class. Rol: DTO / request / response. Paquete: com.codigo.spring.response.

- Transporta datos entre capas o hacia/desde HTTP sin exponer directamente la entidad.
- Sus datos/dependencias clave son: int code, String message, Optional<T> data.
- Anotaciones importantes: @AllArgsConstructor, @Getter, @Setter.
- Analogia: Como un formulario: solo lleva los datos necesarios para una operacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

VueloResponse

Archivo: aerolinea-api-spring-boot/src/main/java/com/codigo/spring/response/VueloResponse.java

Tipo: class. Rol: DTO / request / response. Paquete: com.codigo.spring.response.

- Transporta datos entre capas o hacia/desde HTTP sin exponer directamente la entidad.
- Sus datos/dependencias clave son: LocalDate fechaSalida, LocalDate fechaLlegada, String origen, String destino, AvionResponseBase avion, List<String> pilotosVuelo.
- Anotaciones importantes: @AllArgsConstructor, @Getter, @NoArgsConstructor, @Setter.
- Analogia: Como un formulario: solo lleva los datos necesarios para una operacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

AerolineaService

Archivo: aerolinea-api-spring-boot/src/main/java/com/codigo/spring/service/AerolineaService.java

Tipo: interface. Rol: Contrato de servicio. Paquete: com.codigo.spring.service.

- Define casos de uso disponibles sin revelar la implementacion.
- Analogia: Como el menu de un restaurante: dice que platos existen, no como se cocinan.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

AvionService

Archivo: aerolinea-api-spring-boot/src/main/java/com/codigo/spring/service/AvionService.java

Tipo: interface. Rol: Contrato de servicio. Paquete: com.codigo.spring.service.

- Define casos de uso disponibles sin revelar la implementacion.
- Analogia: Como el menu de un restaurante: dice que platos existen, no como se cocinan.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

BoletoService

Archivo: aerolinea-api-spring-boot/src/main/java/com/codigo/spring/service/BoletoService.java

Tipo: interface. Rol: Contrato de servicio. Paquete: com.codigo.spring.service.

- Define casos de uso disponibles sin revelar la implementacion.
- Analogia: Como el menu de un restaurante: dice que platos existen, no como se cocinan.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

AerolineaServiceImpl

Archivo: aerolinea-api-spring-boot/src/main/java/com/codigo/spring/service/impl/AerolineaServiceImpl.java

Tipo: class. Rol: Servicio de negocio. Paquete: com.codigo.spring.service.impl.

- Implementa reglas, validaciones, llamadas a repositorios o clientes externos.
- Sus datos/dependencias clave son: final AerolineRepository aerolineRepository.
- Metodos para revisar: AerolineEntity save(AerolineEntity aeroline), List<AerolineEntity> findAll(), ResponseBase<AerolineEntity> findById(int id), ResponseBase<AerolineEntity> updateById(AerolineEntity aeroline), AerolineServiceImpl(AerolineRepository aerolineRepository).
- Anotaciones importantes: @Override, @Service.
- Analogia: Como la cocina: combina ingredientes y decide el proceso real.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

AvionServiceImpl

Archivo: aerolinea-api-spring-boot/src/main/java/com/codigo/spring/service/impl/AvionServiceImpl.java

Tipo: class. Rol: Servicio de negocio. Paquete: com.codigo.spring.service.impl.

- Implementa reglas, validaciones, llamadas a repositorios o clientes externos.
- Sus datos/dependencias clave son: final AvionRepository avionRepository, final AerolineRepository aerolineRepository.
- Metodos para revisar: AvionEntity save(AvionEntity avionEntity), ResponseBase<AvionResponse> findById(Integer id), List<AvionResponse> findByModelo(String modelo), List<AvionResponse> findAll(), List<AvionResponse> findAllCapacidad(int min, int max), ResponseBase<AvionResponse> updateAeroline(int idAvion, int idNuevaAeroline), AvionResponse toAvionResponse(AvionEntity avionEntity).
- Anotaciones importantes: @Override, @Service.
- Analogia: Como la cocina: combina ingredientes y decide el proceso real.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

BoletoServiceImpl

Archivo: aerolinea-api-spring-boot/src/main/java/com/codigo/spring/service/impl/BoletoServiceImpl.java

Tipo: class. Rol: Servicio de negocio. Paquete: com.codigo.spring.service.impl.

- Implementa reglas, validaciones, llamadas a repositorios o clientes externos.
- Sus datos/dependencias clave son: final BoletoRepository boletoRepository, final VueloRepository vueloRepository, final PasajeroRepository pasajeroRepository.
- Metodos para revisar: ResponseBase<BoletoResponse> save(BoletoRequest boletoRequest), BoletoServiceImpl(BoletoRepository boletoRepository, VueloRepository vueloRepository, PasajeroRepository pasajeroRepository).
- Anotaciones importantes: @Override, @Service.
- Analogia: Como la cocina: combina ingredientes y decide el proceso real.

- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

PasajeroServiceImpl

Archivo: `aerolinea-api-spring-boot/src/main/java/com/codigo/spring/service/impl/PasajeroServiceImpl.java`

Tipo: class. Rol: Servicio de negocio. Paquete: `com.codigo.spring.service.impl`.

- Implementa reglas, validaciones, llamadas a repositorios o clientes externos.
- Sus datos/dependencias clave son: `final PasajeroRepository pasajeroRepository`, `final BoletoRepository boletoRepository`.
- Metodos para revisar: `ResponseBase<PasajeroResponse> save(PasajeroEntity pasajeroEntity)`, `ResponseBase<PasajeroResponse> findById(Integer id)`, `ResponseBase<PasajeroInfoResponse> findBoleto(Integer id)`, `PasajeroServiceImpl(PasajeroRepository pasajeroRepository, BoletoRepository boletoRepository)`.
- Anotaciones importantes: `@Override`, `@Service`.
- Analogia: Como la cocina: combina ingredientes y decide el proceso real.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

PilotoServiceImpl

Archivo: `aerolinea-api-spring-boot/src/main/java/com/codigo/spring/service/impl/PilotoServiceImpl.java`

Tipo: class. Rol: Servicio de negocio. Paquete: `com.codigo.spring.service.impl`.

- Implementa reglas, validaciones, llamadas a repositorios o clientes externos.
- Sus datos/dependencias clave son: `final PilotoRepository pilotoRepository`.
- Metodos para revisar: `ResponseBase<PilotoResponse> save(PilotoEntity pilotoEntity)`, `ResponseBase<PilotoResponse> findById(Integer id)`, `List<PilotoResponse> findAll()`, `PilotoServiceImpl(PilotoRepository pilotoRepository)`.
- Anotaciones importantes: `@Override`, `@Service`.
- Analogia: Como la cocina: combina ingredientes y decide el proceso real.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

VueloServiceImpl

Archivo: `aerolinea-api-spring-boot/src/main/java/com/codigo/spring/service/impl/VueloServiceImpl.java`

Tipo: class. Rol: Servicio de negocio. Paquete: `com.codigo.spring.service.impl`.

- Implementa reglas, validaciones, llamadas a repositorios o clientes externos.
- Sus datos/dependencias clave son: `final VueloRepository vueloRepository`, `final AvionRepository avionRepository`, `final PilotoRepository pilotoRepository`.
- Metodos para revisar: `ResponseBase<VueloResponse> save(VueloRequest vueloRequest)`, `ResponseBase<VueloResponse> findById(Long id)`, `List<VueloResponse> findByFechaSalida(LocalDate fechaSalida)`, `ResponseBase<VueloResponse> addPilotosToVuelo(VueloRequestUpdatePilotos vueloRequestUpdatePilotos)`, `List<String> processPilotos(List<PilotoEntity> pilotoEntities)`, `VueloResponse toVueloResponse(VueloEntity vueloEntity)`, `VueloServiceImpl(VueloRepository vueloRepository, AvionRepository avionRepository, PilotoRepository pilotoRepository)`.
- Anotaciones importantes: `@Override`, `@Service`.
- Analogia: Como la cocina: combina ingredientes y decide el proceso real.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

PasajeroService

Archivo: `aerolinea-api-spring-boot/src/main/java/com/codigo/spring/service/PasajeroService.java`

Tipo: interface. Rol: Contrato de servicio. Paquete: `com.codigo.spring.service`.

- Define casos de uso disponibles sin revelar la implementacion.
- Analogia: Como el menu de un restaurante: dice que platos existen, no como se cocinan.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

PilotoService

Archivo: `aerolinea-api-spring-boot/src/main/java/com/codigo/spring/service/PilotoService.java`

Tipo: interface. Rol: Contrato de servicio. Paquete: `com.codigo.spring.service`.

- Define casos de uso disponibles sin revelar la implementacion.
- Analogia: Como el menu de un restaurante: dice que platos existen, no como se cocinan.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

VueloService

Archivo: `aerolinea-api-spring-boot/src/main/java/com/codigo/spring/service/VueloService.java`

Tipo: interface. Rol: Contrato de servicio. Paquete: `com.codigo.spring.service`.

- Define casos de uso disponibles sin revelar la implementacion.
- Analogia: Como el menu de un restaurante: dice que platos existen, no como se cocinan.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

Constants

Archivo: `aerolinea-api-spring-boot/src/main/java/com/codigo/spring/utils/Constants.java`

Tipo: class. Rol: Constantes. Paquete: `com.codigo.spring.utils`.

- Centraliza textos, codigos o valores compartidos.
- Sus datos/dependencias clave son: `int CODE_BAD_REQUEST`, `int CODE_NOT_FOUND`, `int CODE_ALREADY_EXISTS`, `int CODE_ALREADY_DELETED`, `int CODE_SUCCESS`, `int CODE_ERROR`.
- Analogia: Como una libreta de etiquetas oficiales para no escribir lo mismo a mano.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

ApplicationTests

Archivo: `aerolinea-api-spring-boot/src/test/java/com/codigo/spring/ApplicationTests.java`

Tipo: class. Rol: Prueba. Paquete: `com.codigo.spring`.

- Verifica comportamiento de una clase, endpoint o contexto.
- Anotaciones importantes: `@SpringBootTest`, `@Test`.
- Analogia: Como una lista de chequeo antes de despegar.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

Preguntas para estudiar este proyecto

- Que clase recibe la solicitud inicial?
- Donde se aplica la regla de negocio?
- Que objeto representa la tabla, documento o dato principal?
- Que DTO evita exponer detalles internos?
- Que test confirma que el comportamiento se mantiene?

Gestion academica Spring Boot

Tema principal: CRUD academico, matriculas, cursos, profesores, estudiantes, validaciones y relaciones JPA.

Analogia del proyecto: Es como una secretaria academica digital: registra alumnos, docentes, cursos y matriculas sin mezclar responsabilidades.

Dependencias y piezas visibles

Modulo Maven: spring-boot-starter-parent, lombok, spring-boot-starter-data-jpa, spring-boot-starter-test, spring-boot-starter-web

Flujo mental

HTTP -> Controller -> Service -> Repository/Ciente externo -> Entity/DTO -> Response.

Clases y archivos explicados

Application

Archivo: gestion-academica-api-spring-boot/src/main/java/com/codigo/spring/Application.java

Tipo: class. Rol: Punto de arranque. Paquete: com.codigo.spring.

- Enciende Spring Boot, escanea componentes y levanta el servidor embebido.
- Metodos para revisar: void main(String[] args).
- Anotaciones importantes: @SpringBootApplication.
- Analogia: Como girar la llave de contacto de un auto.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

OpenApiConfig

Archivo: gestion-academica-api-spring-boot/src/main/java/com/codigo/spring/config/OpenApiConfig.java

Tipo: class. Rol: Configuracion. Paquete: com.codigo.spring.config.

- Declara beans, filtros, clientes o ajustes para que Spring los administre.
- Metodos para revisar: OpenAPI gestionAcademicaOpenApi().
- Anotaciones importantes: @Bean, @Configuration, @tecsup.
- Analogia: Como el tablero de interruptores que conecta piezas del sistema.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

AsignaturaController

Archivo: gestion-academica-api-spring-boot/src/main/java/com/codigo/spring/controller/AsignaturaController.java

Tipo: class. Rol: Controlador REST. Paquete: com.codigo.spring.controller.

- Recibe solicitudes HTTP, toma parametros/body y delega al servicio.
- Expone rutas HTTP como @RequestMapping("/api/v1/asignatura"), @PostMapping("/save"), @GetMapping("/get/{id}").
- Sus datos/dependencias clave son: final AsignaturaService asignaturaService.
- Metodos para revisar: ResponseBase<AsignaturaResponse> save(@Valid @RequestBody AsignaturaRequest request), ResponseBase<AsignaturaResponse> getByld(@PathVariable int id), AsignaturaController(AsignaturaService asignaturaService).
- Anotaciones importantes: @GetMapping, @Operation, @PathVariable, @PostMapping, @RequestBody, @RequestMapping, @RestController, @Tag.
- Analogia: Como una ventanilla: escucha al usuario y pasa el pedido a la oficina correcta.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

CursoController

Archivo: gestion-academica-api-spring-boot/src/main/java/com/codigo/spring/controller/CursoController.java

Tipo: class. Rol: Controlador REST. Paquete: com.codigo.spring.controller.

- Recibe solicitudes HTTP, toma parametros/body y delega al servicio.
- Expone rutas HTTP como `@RequestMapping("/api/v1/curso")`, `@PostMapping("/save")`, `@GetMapping("/get/{id}")`, `@PutMapping("/update/{id}")`.
- Sus datos/dependencias clave son: final `CursoService cursoService`.
- Metodos para revisar: `ResponseBase<CursoResponse> saveCurso(@Valid @RequestBody CursoRequest request)`, `ResponseBase<CursoResponse> getByld(@PathVariable int id)`, `ResponseBase<CursoResponse> updateCurso(@PathVariable int id, @Valid @RequestBody CursoUpdateRequest request)`, `CursoController(CursoService cursoService)`.
- Anotaciones importantes: `@GetMapping`, `@Operation`, `@PathVariable`, `@PostMapping`, `@PutMapping`, `@RequestBody`, `@RequestMapping`, `@RestController`.
- Analogia: Como una ventanilla: escucha al usuario y pasa el pedido a la oficina correcta.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

EstudianteController

Archivo: `gestion-academica-api-spring-boot/src/main/java/com/codigo/spring/controller/EstudianteController.java`

Tipo: class. Rol: Controlador REST. Paquete: `com.codigo.spring.controller`.

- Recibe solicitudes HTTP, toma parametros/body y delega al servicio.
- Expone rutas HTTP como `@RequestMapping("/api/v1/estudiante")`, `@PostMapping("/save")`, `@GetMapping("/get/{dni}")`.
- Sus datos/dependencias clave son: final `EstudianteService estudianteService`.
- Metodos para revisar: `ResponseBase<EstudianteResponse> save(@Valid @RequestBody EstudianteRequest request)`, `ResponseBase<EstudianteResponse> getByld(@PathVariable String dni)`, `EstudianteController(EstudianteService estudianteService)`.
- Anotaciones importantes: `@GetMapping`, `@Operation`, `@PathVariable`, `@PostMapping`, `@RequestBody`, `@RequestMapping`, `@RestController`, `@Tag`.
- Analogia: Como una ventanilla: escucha al usuario y pasa el pedido a la oficina correcta.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

MatriculaController

Archivo: `gestion-academica-api-spring-boot/src/main/java/com/codigo/spring/controller/MatriculaController.java`

Tipo: class. Rol: Controlador REST. Paquete: `com.codigo.spring.controller`.

- Recibe solicitudes HTTP, toma parametros/body y delega al servicio.
- Expone rutas HTTP como `@RequestMapping("/api/v1/matricula")`, `@PostMapping("/save")`, `@GetMapping("/get/{id}")`.
- Sus datos/dependencias clave son: final `MatriculaService matriculaService`.
- Metodos para revisar: `ResponseBase<MatriculaResponse> save(@Valid @RequestBody MatriculaRequest request)`, `ResponseBase<MatriculaResponse> getByld(@PathVariable int id)`, `MatriculaController(MatriculaService matriculaService)`.
- Anotaciones importantes: `@GetMapping`, `@Operation`, `@PathVariable`, `@PostMapping`, `@RequestBody`, `@RequestMapping`, `@RestController`, `@Tag`.
- Analogia: Como una ventanilla: escucha al usuario y pasa el pedido a la oficina correcta.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

ProfesorController

Archivo: `gestion-academica-api-spring-boot/src/main/java/com/codigo/spring/controller/ProfesorController.java`

Tipo: class. Rol: Controlador REST. Paquete: `com.codigo.spring.controller`.

- Recibe solicitudes HTTP, toma parametros/body y delega al servicio.
- Expone rutas HTTP como `@RequestMapping("/api/v1/profesor")`, `@PostMapping("/save")`, `@GetMapping("/get/{dni}")`.
- Sus datos/dependencias clave son: final `ProfesorService profesorService`.
- Metodos para revisar: `ResponseBase<ProfesorResponse> save(@Valid @RequestBody ProfesorRequest request)`, `ResponseBase<ProfesorResponse> getByDni(@PathVariable String dni)`, `ProfesorController(ProfesorService profesorService)`.

- Anotaciones importantes: @GetMapping, @Operation, @PathVariable, @PostMapping, @RequestBody, @RequestMapping, @RestController, @Tag.

- Analogia: Como una ventanilla: escucha al usuario y pasa el pedido a la oficina correcta.

- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

AsignaturaEntity

Archivo: gestion-academica-api-spring-boot/src/main/java/com/codigo/spring/entity/AsignaturaEntity.java

Tipo: class. Rol: Entidad / modelo persistente. Paquete: com.codigo.spring.entity.

- Representa datos de dominio y normalmente se mapea a tablas o documentos.

- Sus datos/dependencias clave son: int id, String nombre, String descripcion.

- Anotaciones importantes: @Entity, @GeneratedValue, @Getter, @Id, @Setter, @Table.

- Analogia: Como una ficha oficial con campos que describen algo del negocio.

- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

CursoEntity

Archivo: gestion-academica-api-spring-boot/src/main/java/com/codigo/spring/entity/CursoEntity.java

Tipo: class. Rol: Entidad / modelo persistente. Paquete: com.codigo.spring.entity.

- Representa datos de dominio y normalmente se mapea a tablas o documentos.

- Sus datos/dependencias clave son: int id, String nombre, String descripcion, ProfesorEntity profesor, AsignaturaEntity asignatura.

- Anotaciones importantes: @Entity, @GeneratedValue, @Getter, @Id, @JoinColumn, @ManyToOne, @Setter, @Table.

- Analogia: Como una ficha oficial con campos que describen algo del negocio.

- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

EstudianteEntity

Archivo: gestion-academica-api-spring-boot/src/main/java/com/codigo/spring/entity/EstudianteEntity.java

Tipo: class. Rol: Entidad / modelo persistente. Paquete: com.codigo.spring.entity.

- Representa datos de dominio y normalmente se mapea a tablas o documentos.

- Sus datos/dependencias clave son: String dni, String nombre, String apellido, String email, LocalDate fechaNacimiento.

- Anotaciones importantes: @Column, @Entity, @Getter, @Id, @Setter, @Table.

- Analogia: Como una ficha oficial con campos que describen algo del negocio.

- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

MatriculaEntity

Archivo: gestion-academica-api-spring-boot/src/main/java/com/codigo/spring/entity/MatriculaEntity.java

Tipo: class. Rol: Entidad / modelo persistente. Paquete: com.codigo.spring.entity.

- Representa datos de dominio y normalmente se mapea a tablas o documentos.

- Sus datos/dependencias clave son: int matriculaId, EstudianteEntity estudiante, CursoEntity curso, LocalDate fechaInscripcion.

- Anotaciones importantes: @Column, @Entity, @GeneratedValue, @Getter, @Id, @JoinColumn, @ManyToOne, @Setter.

- Analogia: Como una ficha oficial con campos que describen algo del negocio.

- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

ProfesorEntity

Archivo: gestion-academica-api-spring-boot/src/main/java/com/codigo/spring/entity/ProfesorEntity.java

Tipo: class. Rol: Entidad / modelo persistente. Paquete: com.codigo.spring.entity.

- Representa datos de dominio y normalmente se mapea a tablas o documentos.

- Sus datos/dependencias clave son: String dni, String nombre, String apellido, String email.

- Anotaciones importantes: @Column, @Entity, @Getter, @Id, @NoArgsConstructor, @Setter, @Table.

- Analogia: Como una ficha oficial con campos que describen algo del negocio.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

GlobalExceptionHandler

Archivo: `gestion-academica-api-spring-boot/src/main/java/com/codigo/spring/exception/GlobalExceptionHandler.java`

Tipo: class. Rol: Excepcion / handler. Paquete: `com.codigo.spring.exception`.

- Representa errores o traduce excepciones en respuestas HTTP comprensibles.
- Metodos para revisar: `ResponseBase<Object> handleValidationException(MethodArgumentNotValidException exception)`, `ResponseBase<Object> handleHttpRequestNotReadableException()`, `ResponseBase<Object> handleMethodArgumentTypeMismatchException(MethodArgumentTypeMismatchException exception)`, `ResponseBase<Object> handleMissingPathVariableException(MissingPathVariableException exception)`, `ResponseBase<Object> handleDataIntegrityViolationException()`, `ResponseBase<Object> handleException()`.
- Anotaciones importantes: `@ExceptionHandler`, `@ResponseStatus`, `@RestControllerAdvice`.
- Analogia: Como un protocolo de emergencia que evita respuestas desordenadas.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

AsignaturaRepository

Archivo: `gestion-academica-api-spring-boot/src/main/java/com/codigo/spring/repository/AsignaturaRepository.java`

Tipo: interface. Rol: Repositorio de datos. Paquete: `com.codigo.spring.repository`.

- Abstrae la persistencia y permite consultar o guardar entidades.
- Analogia: Como el archivo o almacen donde se guarda informacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

CursoRepository

Archivo: `gestion-academica-api-spring-boot/src/main/java/com/codigo/spring/repository/CursoRepository.java`

Tipo: interface. Rol: Repositorio de datos. Paquete: `com.codigo.spring.repository`.

- Abstrae la persistencia y permite consultar o guardar entidades.
- Analogia: Como el archivo o almacen donde se guarda informacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

EstudianteRepository

Archivo: `gestion-academica-api-spring-boot/src/main/java/com/codigo/spring/repository/EstudianteRepository.java`

Tipo: interface. Rol: Repositorio de datos. Paquete: `com.codigo.spring.repository`.

- Abstrae la persistencia y permite consultar o guardar entidades.
- Analogia: Como el archivo o almacen donde se guarda informacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

MatriculaRepository

Archivo: `gestion-academica-api-spring-boot/src/main/java/com/codigo/spring/repository/MatriculaRepository.java`

Tipo: interface. Rol: Repositorio de datos. Paquete: `com.codigo.spring.repository`.

- Abstrae la persistencia y permite consultar o guardar entidades.
- Analogia: Como el archivo o almacen donde se guarda informacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

ProfesorRepository

Archivo: `gestion-academica-api-spring-boot/src/main/java/com/codigo/spring/repository/ProfesorRepository.java`

Tipo: interface. Rol: Repositorio de datos. Paquete: `com.codigo.spring.repository`.

- Abstrae la persistencia y permite consultar o guardar entidades.
- Analogia: Como el archivo o almacen donde se guarda informacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

AsignaturaRequest

Archivo: gestion-academica-api-spring-boot/src/main/java/com/codigo/spring/request/AsignaturaRequest.java

Tipo: class. Rol: DTO / request / response. Paquete: com.codigo.spring.request.

- Transporta datos entre capas o hacia/desde HTTP sin exponer directamente la entidad.
- Sus datos/dependencias clave son: String nombre, String descripcion.
- Anotaciones importantes: @Getter, @NotBlank, @Setter, @Size.
- Analogia: Como un formulario: solo lleva los datos necesarios para una operacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

CursoRequest

Archivo: gestion-academica-api-spring-boot/src/main/java/com/codigo/spring/request/CursoRequest.java

Tipo: class. Rol: DTO / request / response. Paquete: com.codigo.spring.request.

- Transporta datos entre capas o hacia/desde HTTP sin exponer directamente la entidad.
- Sus datos/dependencias clave son: String nombre, String descripcion, String profesor, Integer asignatura.
- Anotaciones importantes: @Getter, @NotBlank, @NotNull, @Setter, @Size.
- Analogia: Como un formulario: solo lleva los datos necesarios para una operacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

CursoUpdateRequest

Archivo: gestion-academica-api-spring-boot/src/main/java/com/codigo/spring/request/CursoUpdateRequest.java

Tipo: class. Rol: DTO / request / response. Paquete: com.codigo.spring.request.

- Transporta datos entre capas o hacia/desde HTTP sin exponer directamente la entidad.
- Sus datos/dependencias clave son: String profesor, Integer asignatura.
- Anotaciones importantes: @Getter, @NotBlank, @NotNull, @Setter.
- Analogia: Como un formulario: solo lleva los datos necesarios para una operacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

EstudianteRequest

Archivo: gestion-academica-api-spring-boot/src/main/java/com/codigo/spring/request/EstudianteRequest.java

Tipo: class. Rol: DTO / request / response. Paquete: com.codigo.spring.request.

- Transporta datos entre capas o hacia/desde HTTP sin exponer directamente la entidad.
- Sus datos/dependencias clave son: String dni, String nombre, String apellido, String email, LocalDate fechaNacimiento.
- Anotaciones importantes: @Email, @Getter, @NotBlank, @NotNull, @Past, @Setter, @Size.
- Analogia: Como un formulario: solo lleva los datos necesarios para una operacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

MatriculaRequest

Archivo: gestion-academica-api-spring-boot/src/main/java/com/codigo/spring/request/MatriculaRequest.java

Tipo: class. Rol: DTO / request / response. Paquete: com.codigo.spring.request.

- Transporta datos entre capas o hacia/desde HTTP sin exponer directamente la entidad.
- Sus datos/dependencias clave son: String estudiante, Integer curso.
- Anotaciones importantes: @Getter, @NotBlank, @NotNull, @Setter.
- Analogia: Como un formulario: solo lleva los datos necesarios para una operacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

ProfesorRequest

Archivo: gestion-academica-api-spring-boot/src/main/java/com/codigo/spring/request/ProfesorRequest.java

Tipo: class. Rol: DTO / request / response. Paquete: com.codigo.spring.request.

- Transporta datos entre capas o hacia/desde HTTP sin exponer directamente la entidad.
- Sus datos/dependencias clave son: String dni, String nombre, String apellido, String email.
- Anotaciones importantes: @Email, @Getter, @NotBlank, @Setter, @Size.
- Analogia: Como un formulario: solo lleva los datos necesarios para una operacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

AsignaturaResponse

Archivo: gestion-academica-api-spring-boot/src/main/java/com/codigo/spring/response/AsignaturaResponse.java

Tipo: class. Rol: DTO / request / response. Paquete: com.codigo.spring.response.

- Transporta datos entre capas o hacia/desde HTTP sin exponer directamente la entidad.
- Sus datos/dependencias clave son: Integer id, String nombre, String descripcion.
- Anotaciones importantes: @AllArgsConstructor, @Getter, @Setter.
- Analogia: Como un formulario: solo lleva los datos necesarios para una operacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

CursoResponse

Archivo: gestion-academica-api-spring-boot/src/main/java/com/codigo/spring/response/CursoResponse.java

Tipo: class. Rol: DTO / request / response. Paquete: com.codigo.spring.response.

- Transporta datos entre capas o hacia/desde HTTP sin exponer directamente la entidad.
- Sus datos/dependencias clave son: String nombre, String descripcion, String profesor, String asignatura.
- Anotaciones importantes: @AllArgsConstructor, @Getter, @Setter.
- Analogia: Como un formulario: solo lleva los datos necesarios para una operacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

EstudianteResponse

Archivo: gestion-academica-api-spring-boot/src/main/java/com/codigo/spring/response/EstudianteResponse.java

Tipo: class. Rol: DTO / request / response. Paquete: com.codigo.spring.response.

- Transporta datos entre capas o hacia/desde HTTP sin exponer directamente la entidad.
- Sus datos/dependencias clave son: String dni, String nombre, String apellido, String email.
- Anotaciones importantes: @AllArgsConstructor, @Getter, @Setter.
- Analogia: Como un formulario: solo lleva los datos necesarios para una operacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

MatriculaResponse

Archivo: gestion-academica-api-spring-boot/src/main/java/com/codigo/spring/response/MatriculaResponse.java

Tipo: class. Rol: DTO / request / response. Paquete: com.codigo.spring.response.

- Transporta datos entre capas o hacia/desde HTTP sin exponer directamente la entidad.
- Sus datos/dependencias clave son: String estudiante, String curso, LocalDate fecha.
- Anotaciones importantes: @AllArgsConstructor, @Getter, @Setter.
- Analogia: Como un formulario: solo lleva los datos necesarios para una operacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

ProfesorResponse

Archivo: gestion-academica-api-spring-boot/src/main/java/com/codigo/spring/response/ProfesorResponse.java

Tipo: class. Rol: DTO / request / response. Paquete: com.codigo.spring.response.

- Transporta datos entre capas o hacia/desde HTTP sin exponer directamente la entidad.
- Sus datos/dependencias clave son: String dni, String nombre, String apellido, String email.
- Anotaciones importantes: @AllArgsConstructor, @Getter, @Setter.

- Analogia: Como un formulario: solo lleva los datos necesarios para una operacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

ResponseBase

Archivo: `gestion-academica-api-spring-boot/src/main/java/com/codigo/spring/response/ResponseBase.java`

Tipo: class. Rol: DTO / request / response. Paquete: `com.codigo.spring.response`.

- Transporta datos entre capas o hacia/desde HTTP sin exponer directamente la entidad.
- Sus datos/dependencias clave son: `int code`, `String message`, `T data`.
- Anotaciones importantes: `@AllArgsConstructor`, `@Getter`, `@NoArgsConstructor`, `@Setter`.
- Analogia: Como un formulario: solo lleva los datos necesarios para una operacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

AsignaturaService

Archivo: `gestion-academica-api-spring-boot/src/main/java/com/codigo/spring/service/AsignaturaService.java`

Tipo: interface. Rol: Contrato de servicio. Paquete: `com.codigo.spring.service`.

- Define casos de uso disponibles sin revelar la implementacion.
- Analogia: Como el menu de un restaurante: dice que platos existen, no como se cocinan.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

CursoService

Archivo: `gestion-academica-api-spring-boot/src/main/java/com/codigo/spring/service/CursoService.java`

Tipo: interface. Rol: Contrato de servicio. Paquete: `com.codigo.spring.service`.

- Define casos de uso disponibles sin revelar la implementacion.
- Analogia: Como el menu de un restaurante: dice que platos existen, no como se cocinan.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

EstudianteService

Archivo: `gestion-academica-api-spring-boot/src/main/java/com/codigo/spring/service/EstudianteService.java`

Tipo: interface. Rol: Contrato de servicio. Paquete: `com.codigo.spring.service`.

- Define casos de uso disponibles sin revelar la implementacion.
- Analogia: Como el menu de un restaurante: dice que platos existen, no como se cocinan.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

AsignaturaServiceImpl

Archivo:
`gestion-academica-api-spring-boot/src/main/java/com/codigo/spring/service/impl/AsignaturaServiceImpl.java`

Tipo: class. Rol: Servicio de negocio. Paquete: `com.codigo.spring.service.impl`.

- Implementa reglas, validaciones, llamadas a repositorios o clientes externos.
- Sus datos/dependencias clave son: `final AsignaturaRepository asignaturaRepository`.
- Metodos para revisar: `AsignaturaResponse save(AsignaturaRequest request)`, `Optional<AsignaturaResponse> getById(Integer id)`, `AsignaturaResponse toResponse(AsignaturaEntity asignatura)`, `AsignaturaServiceImpl(AsignaturaRepository asignaturaRepository)`.
- Anotaciones importantes: `@Override`, `@Service`.
- Analogia: Como la cocina: combina ingredientes y decide el proceso real.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

CursoServiceImpl

Archivo: `gestion-academica-api-spring-boot/src/main/java/com/codigo/spring/service/impl/CursoServiceImpl.java`

Tipo: class. Rol: Servicio de negocio. Paquete: `com.codigo.spring.service.impl`.

- Implementa reglas, validaciones, llamadas a repositorios o clientes externos.

- Sus datos/dependencias clave son: final CursoRepository cursoRepository, final ProfesorRepository profesorRepository, final AsignaturaRepository asignaturaRepository.
- Metodos para revisar: ResponseBase<CursoResponse> save(CursoRequest request), ResponseBase<CursoResponse> getById(Integer id), ResponseBase<CursoResponse> update(Integer id, CursoUpdateRequest request), CursoResponse toResponse(CursoEntity curso), CursoServiceImpl(CursoRepository cursoRepository, ProfesorRepository profesorRepository, AsignaturaRepository asignaturaRepository).
- Anotaciones importantes: @Override, @Service.
- Analogia: Como la cocina: combina ingredientes y decide el proceso real.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

EstudianteServiceImpl

Archivo:
gestion-academica-api-spring-boot/src/main/java/com/codigo/spring/service/impl/EstudianteServiceImpl.java

Tipo: class. Rol: Servicio de negocio. Paquete: com.codigo.spring.service.impl.

- Implementa reglas, validaciones, llamadas a repositorios o clientes externos.
- Sus datos/dependencias clave son: final EstudianteRepository estudianteRepository.
- Metodos para revisar: EstudianteResponse save(EstudianteRequest request), Optional<EstudianteResponse> getById(String dni), EstudianteResponse toResponse(EstudianteEntity estudiante), EstudianteServiceImpl(EstudianteRepository estudianteRepository).
- Anotaciones importantes: @Override, @Service.
- Analogia: Como la cocina: combina ingredientes y decide el proceso real.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

MatriculaServiceImpl

Archivo:
gestion-academica-api-spring-boot/src/main/java/com/codigo/spring/service/impl/MatriculaServiceImpl.java

Tipo: class. Rol: Servicio de negocio. Paquete: com.codigo.spring.service.impl.

- Implementa reglas, validaciones, llamadas a repositorios o clientes externos.
- Sus datos/dependencias clave son: final MatriculaRepository matriculaRepository, final EstudianteRepository estudianteRepository, final CursoRepository cursoRepository.
- Metodos para revisar: ResponseBase<MatriculaResponse> save(MatriculaRequest request), ResponseBase<MatriculaResponse> getById(Integer id), MatriculaResponse toResponse(MatriculaEntity matricula), MatriculaServiceImpl(MatriculaRepository matriculaRepository, EstudianteRepository estudianteRepository, CursoRepository cursoRepository).
- Anotaciones importantes: @Override, @Service.
- Analogia: Como la cocina: combina ingredientes y decide el proceso real.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

ProfesorServiceImpl

Archivo:
gestion-academica-api-spring-boot/src/main/java/com/codigo/spring/service/impl/ProfesorServiceImpl.java

Tipo: class. Rol: Servicio de negocio. Paquete: com.codigo.spring.service.impl.

- Implementa reglas, validaciones, llamadas a repositorios o clientes externos.
- Sus datos/dependencias clave son: final ProfesorRepository repository.
- Metodos para revisar: ProfesorResponse save(ProfesorRequest request), Optional<ProfesorResponse> getById(String dni), ProfesorResponse toResponse(ProfesorEntity profesor), ProfesorServiceImpl(ProfesorRepository repository).
- Anotaciones importantes: @Override, @Service.
- Analogia: Como la cocina: combina ingredientes y decide el proceso real.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

MatriculaService

Archivo: `gestion-academica-api-spring-boot/src/main/java/com/codigo/spring/service/MatriculaService.java`

Tipo: interface. Rol: Contrato de servicio. Paquete: `com.codigo.spring.service`.

- Define casos de uso disponibles sin revelar la implementacion.
- Analogia: Como el menu de un restaurante: dice que platos existen, no como se cocinan.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

ProfesorService

Archivo: `gestion-academica-api-spring-boot/src/main/java/com/codigo/spring/service/ProfesorService.java`

Tipo: interface. Rol: Contrato de servicio. Paquete: `com.codigo.spring.service`.

- Define casos de uso disponibles sin revelar la implementacion.
- Analogia: Como el menu de un restaurante: dice que platos existen, no como se cocinan.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

Constants

Archivo: `gestion-academica-api-spring-boot/src/main/java/com/codigo/spring/utils/Constants.java`

Tipo: class. Rol: Constantes. Paquete: `com.codigo.spring.utils`.

- Centraliza textos, codigos o valores compartidos.
- Sus datos/dependencias clave son: `int CODE_NOT_FOUND`, `int CODE_ALREADY_EXISTS`, `int CODE_ALREADY_DELETED`, `int CODE_SUCESS`, `String MESSAGE_NOT_FOUND`, `String MESSAGE_ALREADY_EXISTS`.
- Analogia: Como una libreta de etiquetas oficiales para no escribir lo mismo a mano.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

ApplicationTests

Archivo: `gestion-academica-api-spring-boot/src/test/java/com/codigo/spring/ApplicationTests.java`

Tipo: class. Rol: Prueba. Paquete: `com.codigo.spring`.

- Verifica comportamiento de una clase, endpoint o contexto.
- Anotaciones importantes: `@SpringBootTest`, `@Test`.
- Analogia: Como una lista de chequeo antes de despegar.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

Preguntas para estudiar este proyecto

- Que clase recibe la solicitud inicial?
- Donde se aplica la regla de negocio?
- Que objeto representa la tabla, documento o dato principal?
- Que DTO evita exponer detalles internos?
- Que test confirma que el comportamiento se mantiene?

OpenFeign RENIEC Spring Boot

Tema principal: Consumo de API externa RENIEC con OpenFeign, DTOs, errores controlados y persistencia.

Analogia del proyecto: OpenFeign funciona como un traductor automatico: tu defines una interfaz Java y el traductor sabe llamar al servicio externo.

Dependencias y piezas visibles

Modulo Maven: spring-boot-starter-parent, lombok, spring-boot-starter-data-jpa, spring-boot-starter-test, spring-boot-starter-web, spring-cloud-starter-openfeign

Flujo mental

HTTP -> Controller -> Service -> Repository/Ciente externo -> Entity/DTO -> Response.

Clases y archivos explicados

Constants

Archivo: openfeign-decolecta-reniec-api-spring-boot/src/main/java/com/codigo/apis_externas/aggregates/constants/Constants.java

Tipo: class. Rol: DTO / request / response. Paquete: com.codigo.apis_externas.aggregates.constants.

- Transporta datos entre capas o hacia/desde HTTP sin exponer directamente la entidad.
- Sus datos/dependencias clave son: String STATUS_ACTIVE, String STATUS_INACTIVE, String USER_ADMIN.
- Analogia: Como un formulario: solo lleva los datos necesarios para una operacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

ApiErrorResponse

Archivo: openfeign-decolecta-reniec-api-spring-boot/src/main/java/com/codigo/apis_externas/aggregates/response/ApiErrorResponse.java

Tipo: class. Rol: DTO / request / response. Paquete: com.codigo.apis_externas.aggregates.response.

- Transporta datos entre capas o hacia/desde HTTP sin exponer directamente la entidad.
- Sus datos/dependencias clave son: String error, String message.
- Anotaciones importantes: @AllArgsConstructor, @Getter, @NoArgsConstructor, @Setter.
- Analogia: Como un formulario: solo lleva los datos necesarios para una operacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

ReniecResponse

Archivo: openfeign-decolecta-reniec-api-spring-boot/src/main/java/com/codigo/apis_externas/aggregates/response/ReniecResponse.java

Tipo: class. Rol: DTO / request / response. Paquete: com.codigo.apis_externas.aggregates.response.

- Transporta datos entre capas o hacia/desde HTTP sin exponer directamente la entidad.
- Sus datos/dependencias clave son: String nombres, String apellidoPaterno, String apellidoMaterno, String nombreCompleto, String numeroDocumento.
- Anotaciones importantes: @Getter, @JsonAlias, @Setter.
- Analogia: Como un formulario: solo lleva los datos necesarios para una operacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

ResponseBase

Archivo: openfeign-decolecta-reniec-api-spring-boot/src/main/java/com/codigo/apis_externas/aggregates/response/ResponseBase.java

Tipo: class. Rol: DTO / request / response. Paquete: com.codigo.apis_externas.aggregates.response.

- Transporta datos entre capas o hacia/desde HTTP sin exponer directamente la entidad.
- Sus datos/dependencias clave son: Integer code, String message, Optional<T> entity.

- Anotaciones importantes: @Getter, @Setter.
- Analogia: Como un formulario: solo lleva los datos necesarios para una operacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

ApisExternasApplication

Archivo:
openfeign-decolecta-reniec-api-spring-boot/src/main/java/com/codigo/apis_externas/ApisExternasApplication.java

Tipo: class. Rol: Punto de arranque. Paquete: com.codigo.apis_externas.

- Enciende Spring Boot, escanea componentes y levanta el servidor embebido.
- Metodos para revisar: void main(String[] args).
- Anotaciones importantes: @EnableFeignClients, @SpringBootApplication.
- Analogia: Como girar la llave de contacto de un auto.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

ClientReniec

Archivo:
openfeign-decolecta-reniec-api-spring-boot/src/main/java/com/codigo/apis_externas/client/ClientReniec.java

Tipo: interface. Rol: Cliente externo. Paquete: com.codigo.apis_externas.client.

- Define o ejecuta llamadas HTTP hacia servicios externos.
- Expone rutas HTTP como @GetMapping(value).
- Anotaciones importantes: @FeignClient, @GetMapping, @RequestHeader, @RequestParam.
- Analogia: Como un mensajero que sabe la direccion y el formato del pedido.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

GlobalExceptionHandler

Archivo: openfeign-decolecta-reniec-api-spring-boot/src/main/java/com/codigo/apis_externas/controller/advice/GlobalExceptionHandler.java

Tipo: class. Rol: Controlador REST. Paquete: com.codigo.apis_externas.controller.advice.

- Recibe solicitudes HTTP, toma parametros/body y delega al servicio.
- Metodos para revisar: ResponseEntity<ApiErrorResponse> handleException(Throwable ex).
- Anotaciones importantes: @ExceptionHandler, @RestControllerAdvice.
- Analogia: Como una ventanilla: escucha al usuario y pasa el pedido a la oficina correcta.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

PersonController

Archivo: openfeign-decolecta-reniec-api-spring-boot/src/main/java/com/codigo/apis_externas/controller/PersonController.java

Tipo: class. Rol: Controlador REST. Paquete: com.codigo.apis_externas.controller.

- Recibe solicitudes HTTP, toma parametros/body y delega al servicio.
- Expone rutas HTTP como @RequestMapping("/api/person/"), @GetMapping("/find/{dni}"), @PostMapping("/save/{dni}"), @GetMapping().
- Sus datos/dependencias clave son: final PersonService personService.
- Metodos para revisar: ResponseEntity<ResponseBase<ReniecResponse>> findPerson(@PathVariable String dni), ResponseEntity<ResponseBase<PersonEntity>> savePerson(@PathVariable String dni), ResponseEntity<ResponseBase<List<PersonEntity>>> findAllActive().
- Anotaciones importantes: @GetMapping, @PathVariable, @PostMapping, @RequestMapping, @RequiredArgsConstructor, @RestController.
- Analogia: Como una ventanilla: escucha al usuario y pasa el pedido a la oficina correcta.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

PersonEntity

Archivo: openfeign-decolecta-reniec-api-spring-boot/src/main/java/com/codigo/apis_externas/entity/PersonEntity.java

Tipo: class. Rol: Entidad / modelo persistente. Paquete: com.codigo.apis_externas.entity.

- Representa datos de dominio y normalmente se mapea a tablas o documentos.
- Sus datos/dependencias clave son: Long id, String names, String lastName, String motherLastName, String fullName, String typeDocument.
- Anotaciones importantes: @AllArgsConstructor, @Builder, @Column, @Entity, @GeneratedValue, @Getter, @Id, @NoArgsConstructor.
- Analogia: Como una ficha oficial con campos que describen algo del negocio.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

ConsultaReniecException

Archivo: openfeign-decolecta-reniec-api-spring-boot/src/main/java/com/codigo/apis_externas/exception/ConsultaReniecException.java

Tipo: class. Rol: Excepcion / handler. Paquete: com.codigo.apis_externas.exception.

- Representa errores o traduce excepciones en respuestas HTTP comprensibles.
- Metodos para revisar: ConsultaReniecException(String message), ConsultaReniecException(String message, Throwable cause).
- Analogia: Como un protocolo de emergencia que evita respuestas desordenadas.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

PersonMapper

Archivo: openfeign-decolecta-reniec-api-spring-boot/src/main/java/com/codigo/apis_externas/mapper/PersonMapper.java

Tipo: class. Rol: Mapper. Paquete: com.codigo.apis_externas.mapper.

- Convierte entidades, requests y responses para separar modelos internos y externos.
- Metodos para revisar: PersonEntity fromReniecResponse(ReniecResponse reniecResponse), PersonMapper().
- Analogia: Como un traductor entre el idioma de la base de datos y el idioma de la API.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

PersonRepository

Archivo: openfeign-decolecta-reniec-api-spring-boot/src/main/java/com/codigo/apis_externas/repository/PersonRepository.java

Tipo: interface. Rol: Repositorio de datos. Paquete: com.codigo.apis_externas.repository.

- Abstrae la persistencia y permite consultar o guardar entidades.
- Analogia: Como el archivo o almacen donde se guarda informacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

PersonServiceImpl

Archivo: openfeign-decolecta-reniec-api-spring-boot/src/main/java/com/codigo/apis_externas/service/impl/PersonServiceImpl.java

Tipo: class. Rol: Servicio de negocio. Paquete: com.codigo.apis_externas.service.impl.

- Implementa reglas, validaciones, llamadas a repositorios o clientes externos.
- Sus datos/dependencias clave son: final ClientReniec clientReniec, final PersonRepository personRepository, String token.
- Metodos para revisar: ResponseBase<ReniecResponse> findByDni(String dni), ReniecResponse executionReniec(String dni), ResponseBase<PersonEntity> registerPerson(String dni), ResponseBase<List<PersonEntity>> findPersonActive(), <T> ResponseBase<T> buildResponse(int code, String message, Optional<T> optional).
- Anotaciones importantes: @Log4j2, @Override, @RequiredArgsConstructor, @Service, @Value.
- Analogia: Como la cocina: combina ingredientes y decide el proceso real.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

PersonService

Archivo:

openfeign-decolecta-reniec-api-spring-boot/src/main/java/com/codigo/apis_externas/service/PersonService.java

Tipo: interface. Rol: Contrato de servicio. Paquete: com.codigo.apis_externas.service.

- Define casos de uso disponibles sin revelar la implementacion.
- Analogia: Como el menu de un restaurante: dice que platos existen, no como se cocinan.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

ApisExternasApplicationTests

Archivo: openfeign-decolecta-reniec-api-spring-boot/src/test/java/com/codigo/apis_externas/ApisExternasApplicationTests.java

Tipo: class. Rol: Prueba. Paquete: com.codigo.apis_externas.

- Verifica comportamiento de una clase, endpoint o contexto.
- Anotaciones importantes: @SpringBootTest, @Test.
- Analogia: Como una lista de chequeo antes de despegar.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

Preguntas para estudiar este proyecto

- Que clase recibe la solicitud inicial?
- Donde se aplica la regla de negocio?
- Que objeto representa la tabla, documento o dato principal?
- Que DTO evita exponer detalles internos?
- Que test confirma que el comportamiento se mantiene?

RestTemplate RENIEC Spring Boot

Tema principal: Consumo de API externa RENIEC con RestTemplate, headers, token y manejo de errores.

Analogia del proyecto: RestTemplate es como hacer una llamada telefonica manual: armas la cabecera, marcas la URL y procesas la respuesta.

Dependencias y piezas visibles

Modulo Maven: spring-boot-starter-parent, lombok, spring-boot-starter-test, spring-boot-starter-web

Flujo mental

Clases conectadas por Spring Boot segun rol: configuracion, servicio, modelo y prueba.

Clases y archivos explicados

Constants

Archivo:
restTemplate-reniec-api-spring-boot/src/main/java/com/codigo/restTemplate/aggregates/constants/Constants.java

Tipo: class. Rol: DTO / request / response. Paquete: com.codigo.restTemplate.aggregates.constants.

- Transporta datos entre capas o hacia/desde HTTP sin exponer directamente la entidad.
- Sus datos/dependencias clave son: String BEARER.
- Analogia: Como un formulario: solo lleva los datos necesarios para una operacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

ApiErrorResponse

Archivo: restTemplate-reniec-api-spring-boot/src/main/java/com/codigo/restTemplate/aggregates/response/ApiErrorResponse.java

Tipo: class. Rol: DTO / request / response. Paquete: com.codigo.restTemplate.aggregates.response.

- Transporta datos entre capas o hacia/desde HTTP sin exponer directamente la entidad.
- Sus datos/dependencias clave son: String error, String message.
- Anotaciones importantes: @AllArgsConstructor, @Getter, @NoArgsConstructor, @Setter.
- Analogia: Como un formulario: solo lleva los datos necesarios para una operacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

ReniecResponse

Archivo: restTemplate-reniec-api-spring-boot/src/main/java/com/codigo/restTemplate/aggregates/response/ReniecResponse.java

Tipo: class. Rol: DTO / request / response. Paquete: com.codigo.restTemplate.aggregates.response.

- Transporta datos entre capas o hacia/desde HTTP sin exponer directamente la entidad.
- Sus datos/dependencias clave son: String nombres, String apellidoPaterno, String apellidoMaterno, String nombreCompleto, String numeroDocumento.
- Anotaciones importantes: @Getter, @JsonAlias, @Setter.
- Analogia: Como un formulario: solo lleva los datos necesarios para una operacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

RestTemplateConfig

Archivo:
restTemplate-reniec-api-spring-boot/src/main/java/com/codigo/restTemplate/config/RestTemplateConfig.java

Tipo: class. Rol: Configuracion. Paquete: com.codigo.restTemplate.config.

- Declara beans, filtros, clientes o ajustes para que Spring los administre.
- Metodos para revisar: RestTemplate restTemplate().
- Anotaciones importantes: @Bean, @Configuration.

- Analogia: Como el tablero de interruptores que conecta piezas del sistema.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

GlobalExceptionHandler

Archivo: `restTemplate-reniec-api-spring-boot/src/main/java/com/codigo/restTemplate/controller/advice/GlobalExceptionHandler.java`

Tipo: class. Rol: Controlador REST. Paquete: `com.codigo.restTemplate.controller.advice`.

- Recibe solicitudes HTTP, toma parametros/body y delega al servicio.
- Metodos para revisar: `ResponseEntity<ApiErrorResponse> handleConsultaReniecException(Throwable ex)`.
- Anotaciones importantes: `@ExceptionHandler`, `@RestControllerAdvice`.
- Analogia: Como una ventanilla: escucha al usuario y pasa el pedido a la oficina correcta.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

ReniecController

Archivo: `restTemplate-reniec-api-spring-boot/src/main/java/com/codigo/restTemplate/controller/ReniecController.java`

Tipo: class. Rol: Controlador REST. Paquete: `com.codigo.restTemplate.controller`.

- Recibe solicitudes HTTP, toma parametros/body y delega al servicio.
- Expone rutas HTTP como `@RequestMapping("/api/reniec/v1")`, `@GetMapping("/{dni}")`, `@GetMapping("/v2/{dni}")`.
- Sus datos/dependencias clave son: `final ReniecClientService reniecClient`.
- Metodos para revisar: `ResponseEntity<ReniecResponse> findByDni(@PathVariable String dni)`, `ResponseEntity<ReniecResponse> findByDni2(@PathVariable String dni)`.
- Anotaciones importantes: `@GetMapping`, `@PathVariable`, `@RequestMapping`, `@RequiredArgsConstructor`, `@RestController`.
- Analogia: Como una ventanilla: escucha al usuario y pasa el pedido a la oficina correcta.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

ConsultaReniecException

Archivo: `restTemplate-reniec-api-spring-boot/src/main/java/com/codigo/restTemplate/exception/ConsultaReniecException.java`

Tipo: class. Rol: Excepcion / handler. Paquete: `com.codigo.restTemplate.exception`.

- Representa errores o traduce excepciones en respuestas HTTP comprensibles.
- Metodos para revisar: `ConsultaReniecException(String message)`, `ConsultaReniecException(String message, Throwable cause)`.
- Analogia: Como un protocolo de emergencia que evita respuestas desordenadas.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

RestTemplateApplication

Archivo: `restTemplate-reniec-api-spring-boot/src/main/java/com/codigo/restTemplate/RestTemplateApplication.java`

Tipo: class. Rol: Punto de arranque. Paquete: `com.codigo.restTemplate`.

- Enciende Spring Boot, escanea componentes y levanta el servidor embebido.
- Metodos para revisar: `void main(String[] args)`.
- Anotaciones importantes: `@SpringBootApplication`.
- Analogia: Como girar la llave de contacto de un auto.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

ReniecClientServiceImpl

Archivo: `restTemplate-reniec-api-spring-boot/src/main/java/com/codigo/restTemplate/service/impl/ReniecClientServiceImpl.java`

Tipo: class. Rol: Servicio de negocio. Paquete: `com.codigo.restTemplate.service.impl`.

- Implementa reglas, validaciones, llamadas a repositorios o clientes externos.

- Sus datos/dependencias clave son: final RestTemplate restTemplate, String tokenApi, String reniecUrl.
- Metodos para revisar: Optional<ReniecResponse> getInfoReniec(String dni), HttpHeaders createHeaders().
- Anotaciones importantes: @Log4j2, @Override, @RequiredArgsConstructor, @Service, @Value.
- Analogia: Como la cocina: combina ingredientes y decide el proceso real.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

ReniecClientService

Archivo:
restTemplate-reniec-api-spring-boot/src/main/java/com/codigo/restTemplate/service/ReniecClientService.java

Tipo: interface. Rol: Contrato de servicio. Paquete: com.codigo.restTemplate.service.

- Define casos de uso disponibles sin revelar la implementacion.
- Analogia: Como el menu de un restaurante: dice que platos existen, no como se cocinan.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

RestTemplateApplicationTests

Archivo:
restTemplate-reniec-api-spring-boot/src/test/java/com/codigo/restTemplate/RestTemplateApplicationTests.java

Tipo: class. Rol: Prueba. Paquete: com.codigo.restTemplate.

- Verifica comportamiento de una clase, endpoint o contexto.
- Anotaciones importantes: @SpringBootTest, @Test.
- Analogia: Como una lista de chequeo antes de despegar.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

Preguntas para estudiar este proyecto

- Que clase recibe la solicitud inicial?
- Donde se aplica la regla de negocio?
- Que objeto representa la tabla, documento o dato principal?
- Que DTO evita exponer detalles internos?
- Que test confirma que el comportamiento se mantiene?

Retrofit RENIEC Spring Boot

Tema principal: Cliente Retrofit, mapeo de persona, persistencia y separacion controller-service-repository.

Analogia del proyecto: Retrofit es como un contrato de mensajeria: describes la ruta y el paquete, y el cliente se encarga del viaje HTTP.

Dependencias y piezas visibles

Modulo Maven: spring-boot-starter-parent, lombok, spring-boot-starter-data-jpa, spring-boot-starter-test, spring-boot-starter-web

Flujo mental

HTTP -> Controller -> Service -> Repository/Cliente externo -> Entity/DTO -> Response.

Clases y archivos explicados

ConexionDb

Archivo: retrofit-reniec-api-spring-boot/src/main/java/com/codigo/retrofit/aggregates/ConexionDb.java

Tipo: class. Rol: DTO / request / response. Paquete: com.codigo.retrofit.aggregates.

- Transporta datos entre capas o hacia/desde HTTP sin exponer directamente la entidad.
- Sus datos/dependencias clave son: static ConexionDb conexionDb.
- Metodos para revisar: ConexionDb obtenerInstancia(), ConexionDb().
- Analogia: Como un formulario: solo lleva los datos necesarios para una operacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

Constants

Archivo: retrofit-reniec-api-spring-boot/src/main/java/com/codigo/retrofit/aggregates/constants/Constants.java

Tipo: class. Rol: DTO / request / response. Paquete: com.codigo.retrofit.aggregates.constants.

- Transporta datos entre capas o hacia/desde HTTP sin exponer directamente la entidad.
- Sus datos/dependencias clave son: String STATUS_ACTIVE, String STATUS_INACTIVE, String USER_ADMIN.
- Analogia: Como un formulario: solo lleva los datos necesarios para una operacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

PersonUpdateRequest

Archivo:
retrofit-reniec-api-spring-boot/src/main/java/com/codigo/retrofit/aggregates/request/PersonUpdateRequest.java

Tipo: class. Rol: DTO / request / response. Paquete: com.codigo.retrofit.aggregates.request.

- Transporta datos entre capas o hacia/desde HTTP sin exponer directamente la entidad.
- Sus datos/dependencias clave son: String names, String lastName, String motherLastName, String fullName, String typeDocument, String numberDocument.
- Anotaciones importantes: @Getter, @Setter.
- Analogia: Como un formulario: solo lleva los datos necesarios para una operacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

ApiErrorResponse

Archivo:
retrofit-reniec-api-spring-boot/src/main/java/com/codigo/retrofit/aggregates/response/ApiErrorResponse.java

Tipo: class. Rol: DTO / request / response. Paquete: com.codigo.retrofit.aggregates.response.

- Transporta datos entre capas o hacia/desde HTTP sin exponer directamente la entidad.
- Sus datos/dependencias clave son: String error, String message.
- Anotaciones importantes: @AllArgsConstructor, @Getter, @NoArgsConstructor, @Setter.
- Analogia: Como un formulario: solo lleva los datos necesarios para una operacion.

- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

ReniecResponse

Archivo:

retrofit-reniec-api-spring-boot/src/main/java/com/codigo/retrofit/aggregates/response/ReniecResponse.java

Tipo: class. Rol: DTO / request / response. Paquete: com.codigo.retrofit.aggregates.response.

- Transporta datos entre capas o hacia/desde HTTP sin exponer directamente la entidad.
- Sus datos/dependencias clave son: String nombres, String apellidoPaterno, String apellidoMaterno, String nombreCompleto, String numeroDocumento.
- Anotaciones importantes: @Getter, @SerializedName, @Setter.
- Analogia: Como un formulario: solo lleva los datos necesarios para una operacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

ResponseBase

Archivo:

retrofit-reniec-api-spring-boot/src/main/java/com/codigo/retrofit/aggregates/response/ResponseBase.java

Tipo: class. Rol: DTO / request / response. Paquete: com.codigo.retrofit.aggregates.response.

- Transporta datos entre capas o hacia/desde HTTP sin exponer directamente la entidad.
- Sus datos/dependencias clave son: Integer code, String message, Optional<T> entity.
- Anotaciones importantes: @Getter, @Setter.
- Analogia: Como un formulario: solo lleva los datos necesarios para una operacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

GlobalExceptionHandler

Archivo:

retrofit-reniec-api-spring-boot/src/main/java/com/codigo/retrofit/controller/advice/GlobalExceptionHandler.java

Tipo: class. Rol: Controlador REST. Paquete: com.codigo.retrofit.controller.advice.

- Recibe solicitudes HTTP, toma parametros/body y delega al servicio.
- Metodos para revisar: ResponseEntity<ApiErrorResponse> handleException(Throwable ex).
- Anotaciones importantes: @ExceptionHandler, @RestControllerAdvice.
- Analogia: Como una ventanilla: escucha al usuario y pasa el pedido a la oficina correcta.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

PersonController

Archivo: retrofit-reniec-api-spring-boot/src/main/java/com/codigo/retrofit/controller/PersonController.java

Tipo: class. Rol: Controlador REST. Paquete: com.codigo.retrofit.controller.

- Recibe solicitudes HTTP, toma parametros/body y delega al servicio.
- Expone rutas HTTP como @RequestMapping("/api/person/"), @GetMapping("/find/{dni}"), @PostMapping("/save/{dni}"), @GetMapping().
- Sus datos/dependencias clave son: final PersonService personService.
- Metodos para revisar: ResponseEntity<ReniecResponse> findPerson(@PathVariable String dni), ResponseEntity<ResponseBase<PersonEntity>> savePerson(@PathVariable String dni), ResponseEntity<ResponseBase<List<PersonEntity>>> findAllActive(), ResponseEntity<ResponseBase<PersonEntity>> findById(@PathVariable Long id), ResponseEntity<ResponseBase<PersonEntity>> updatePerson(@PathVariable Long id, @RequestBody PersonUpdateRequest request), ResponseEntity<ResponseBase<PersonEntity>> updateStatus(@PathVariable Long id, @PathVariable String status), ResponseEntity<ResponseBase<PersonEntity>> deletePerson(@PathVariable Long id).
- Anotaciones importantes: @DeleteMapping, @GetMapping, @PathVariable, @PostMapping, @PutMapping, @RequestBody, @RequestMapping, @RequiredArgsConstructor.
- Analogia: Como una ventanilla: escucha al usuario y pasa el pedido a la oficina correcta.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

PersonEntity

Archivo: retrofit-reniec-api-spring-boot/src/main/java/com/codigo/retrofit/entity/PersonEntity.java

Tipo: class. Rol: Entidad / modelo persistente. Paquete: com.codigo.retrofit.entity.

- Representa datos de dominio y normalmente se mapea a tablas o documentos.
- Sus datos/dependencias clave son: Long id, String names, String lastName, String motherLastName, String fullName, String typeDocument.
- Anotaciones importantes: @AllArgsConstructor, @Builder, @Column, @Entity, @GeneratedValue, @Getter, @Id, @NoArgsConstructor.
- Analogia: Como una ficha oficial con campos que describen algo del negocio.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

ConsultaReniecException

Archivo:
retrofit-reniec-api-spring-boot/src/main/java/com/codigo/retrofit/exception/ConsultaReniecException.java

Tipo: class. Rol: Excepcion / handler. Paquete: com.codigo.retrofit.exception.

- Representa errores o traduce excepciones en respuestas HTTP comprensibles.
- Metodos para revisar: ConsultaReniecException(String message), ConsultaReniecException(String message, Throwable cause).
- Analogia: Como un protocolo de emergencia que evita respuestas desordenadas.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

PersonMapper

Archivo: retrofit-reniec-api-spring-boot/src/main/java/com/codigo/retrofit/mapper/PersonMapper.java

Tipo: class. Rol: Mapper. Paquete: com.codigo.retrofit.mapper.

- Convierte entidades, requests y responses para separar modelos internos y externos.
- Metodos para revisar: PersonEntity fromReniecResponse(ReniecResponse reniecResponse), void updateEntity(PersonEntity personEntity, PersonUpdateRequest request), void updateStatus(PersonEntity personEntity, String status), PersonMapper().
- Analogia: Como un traductor entre el idioma de la base de datos y el idioma de la API.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

PersonRepository

Archivo: retrofit-reniec-api-spring-boot/src/main/java/com/codigo/retrofit/repository/PersonRepository.java

Tipo: interface. Rol: Repositorio de datos. Paquete: com.codigo.retrofit.repository.

- Abstrae la persistencia y permite consultar o guardar entidades.
- Analogia: Como el archivo o almacen donde se guarda informacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

ClientReniecService

Archivo: retrofit-reniec-api-spring-boot/src/main/java/com/codigo/retrofit/retrofit/ClientReniecService.java

Tipo: interface. Rol: Cliente externo. Paquete: com.codigo.retrofit.retrofit.

- Define o ejecuta llamadas HTTP hacia servicios externos.
- Anotaciones importantes: @GET, @Header, @Query.
- Analogia: Como un mensajero que sabe la direccion y el formato del pedido.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

ClientRetrofit

Archivo: retrofit-reniec-api-spring-boot/src/main/java/com/codigo/retrofit/retrofit/ClientRetrofit.java

Tipo: class. Rol: Cliente externo. Paquete: com.codigo.retrofit.retrofit.

- Define o ejecuta llamadas HTTP hacia servicios externos.

- Sus datos/dependencias clave son: static Retrofit retrofit.
- Metodos para revisar: Retrofit getRetrofit().
- Analogia: Como un mensajero que sabe la direccion y el formato del pedido.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

RetrofitApplication

Archivo: retrofit-reniec-api-spring-boot/src/main/java/com/codigo/retrofit/RetrofitApplication.java

Tipo: class. Rol: Punto de arranque. Paquete: com.codigo.retrofit.

- Enciende Spring Boot, escanea componentes y levanta el servidor embebido.
- Metodos para revisar: void main(String[] args).
- Anotaciones importantes: @SpringBootApplication.
- Analogia: Como girar la llave de contacto de un auto.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

PersonServiceImpl

Archivo: retrofit-reniec-api-spring-boot/src/main/java/com/codigo/retrofit/service/impl/PersonServiceImpl.java

Tipo: class. Rol: Servicio de negocio. Paquete: com.codigo.retrofit.service.impl.

- Implementa reglas, validaciones, llamadas a repositorios o clientes externos.
- Sus datos/dependencias clave son: final PersonRepository personRepository, String token.
- Metodos para revisar: ReniecResponse findByDni(String dni), ResponseBase<PersonEntity> registerPerson(String dni), ResponseBase<List<PersonEntity>> findPersonActive(), ResponseBase<PersonEntity> findById(Long id), ResponseBase<PersonEntity> updatePerson(Long id, PersonUpdateRequest request), ResponseBase<PersonEntity> updateStatus(Long id, String status), ResponseBase<PersonEntity> deletePerson(Long id).
- Anotaciones importantes: @Log4j2, @Override, @RequiredArgsConstructor, @Service, @Value.
- Analogia: Como la cocina: combina ingredientes y decide el proceso real.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

PersonService

Archivo: retrofit-reniec-api-spring-boot/src/main/java/com/codigo/retrofit/service/PersonService.java

Tipo: interface. Rol: Contrato de servicio. Paquete: com.codigo.retrofit.service.

- Define casos de uso disponibles sin revelar la implementacion.
- Analogia: Como el menu de un restaurante: dice que platos existen, no como se cocinan.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

RetrofitApplicationTests

Archivo: retrofit-reniec-api-spring-boot/src/test/java/com/codigo/retrofit/RetrofitApplicationTests.java

Tipo: class. Rol: Prueba. Paquete: com.codigo.retrofit.

- Verifica comportamiento de una clase, endpoint o contexto.
- Anotaciones importantes: @SpringBootTest, @Test.
- Analogia: Como una lista de chequeo antes de despegar.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

Preguntas para estudiar este proyecto

- Que clase recibe la solicitud inicial?
- Donde se aplica la regla de negocio?
- Que objeto representa la tabla, documento o dato principal?
- Que DTO evita exponer detalles internos?
- Que test confirma que el comportamiento se mantiene?

WebClient RENIEC Spring Boot

Tema principal: Cliente HTTP moderno con WebClient para consumir RENIEC usando configuracion externa.

Analogia del proyecto: WebClient es como pedir por una app moderna: prepara la solicitud, la envia y espera una respuesta de forma flexible.

Dependencias y piezas visibles

Modulo Maven: spring-boot-starter-parent, lombok, spring-boot-starter-test, spring-boot-starter-web

Flujo mental

Clases conectadas por Spring Boot segun rol: configuracion, servicio, modelo y prueba.

Clases y archivos explicados

ReniecController

Archivo: webclient-reniec-api-spring-boot/src/main/java/com/codigo/webClient/controller/ReniecController.java

Tipo: class. Rol: Controlador REST. Paquete: com.codigo.webClient.controller.

- Recibe solicitudes HTTP, toma parametros/body y delega al servicio.
- Expone rutas HTTP como `@RequestMapping("/api/reniec")`, `@GetMapping("/{dni}")`.
- Sus datos/dependencias clave son: `final ReniecService reniecService`.
- Metodos para revisar: `Mono<ReniecResponse> findByDni(@PathVariable String dni)`.
- Anotaciones importantes: `@GetMapping`, `@Log4j2`, `@PathVariable`, `@RequestMapping`, `@RequiredArgsConstructor`, `@RestController`.
- Analogia: Como una ventanilla: escucha al usuario y pasa el pedido a la oficina correcta.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

ReniecResponse

Archivo: webclient-reniec-api-spring-boot/src/main/java/com/codigo/webClient/response/ReniecResponse.java

Tipo: class. Rol: DTO / request / response. Paquete: com.codigo.webClient.response.

- Transporta datos entre capas o hacia/desde HTTP sin exponer directamente la entidad.
- Sus datos/dependencias clave son: `String nombres`, `String apellidoPaterno`, `String apellidoMaterno`, `String nombreCompleto`, `String tipoDocumento`, `String numeroDocumento`.
- Anotaciones importantes: `@AllArgsConstructor`, `@Getter`, `@JsonProperty`, `@NoArgsConstructor`, `@Setter`.
- Analogia: Como un formulario: solo lleva los datos necesarios para una operacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

ReniecServiceImpl

Archivo:
webclient-reniec-api-spring-boot/src/main/java/com/codigo/webClient/service/impl/ReniecServiceImpl.java

Tipo: class. Rol: Servicio de negocio. Paquete: com.codigo.webClient.service.impl.

- Implementa reglas, validaciones, llamadas a repositorios o clientes externos.
- Sus datos/dependencias clave son: `String token`, `String reniecUrl`, `final WebClient.Builder webClientBuilder`.
- Metodos para revisar: `Mono<ReniecResponse> getInfoReniec(String dni)`.
- Anotaciones importantes: `@Log4j2`, `@Override`, `@RequiredArgsConstructor`, `@Service`, `@Value`.
- Analogia: Como la cocina: combina ingredientes y decide el proceso real.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

ReniecService

Archivo: webclient-reniec-api-spring-boot/src/main/java/com/codigo/webClient/service/ReniecService.java

Tipo: interface. Rol: Contrato de servicio. Paquete: com.codigo.webClient.service.

- Define casos de uso disponibles sin revelar la implementacion.
- Analogia: Como el menu de un restaurante: dice que platos existen, no como se cocinan.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

WebClientApplication

Archivo: webclient-reniec-api-spring-boot/src/main/java/com/codigo/webClient/WebClientApplication.java

Tipo: class. Rol: Punto de arranque. Paquete: com.codigo.webClient.

- Enciende Spring Boot, escanea componentes y levanta el servidor embebido.
- Metodos para revisar: void main(String[] args).
- Anotaciones importantes: @SpringBootApplication.
- Analogia: Como girar la llave de contacto de un auto.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

WebClientApplicationTests

Archivo: webclient-reniec-api-spring-boot/src/test/java/com/codigo/webClient/WebClientApplicationTests.java

Tipo: class. Rol: Prueba. Paquete: com.codigo.webClient.

- Verifica comportamiento de una clase, endpoint o contexto.
- Anotaciones importantes: @SpringBootTest, @Test.
- Analogia: Como una lista de chequeo antes de despegar.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

Preguntas para estudiar este proyecto

- Que clase recibe la solicitud inicial?
- Donde se aplica la regla de negocio?
- Que objeto representa la tabla, documento o dato principal?
- Que DTO evita exponer detalles internos?
- Que test confirma que el comportamiento se mantiene?

Hexagonal RENIEC Spring Boot

Tema principal: Arquitectura hexagonal, dominio, puertos, adaptadores, OpenFeign, JPA y Mongo.

Analogía del proyecto: La arquitectura hexagonal es una ciudad amurallada: el dominio vive protegido y los adaptadores son las puertas hacia la base de datos o APIs externas.

Dependencias y piezas visibles

Modulo Maven: ms-register-hexagonal, Modulo Maven: spring-boot-starter-parent, lombok, spring-boot-starter-data-jpa, spring-boot-starter-data-mongodb, spring-boot-starter-test, spring-boot-starter-web, spring-cloud-starter-openfeign

Flujo mental

HTTP -> Controller -> Service -> Repository/Ciente externo -> Entity/DTO -> Response.

Clases y archivos explicados

ApplicationLauncher

Archivo: ms-hexagonal-reniec-openfeign-spring-boot/application/src/main/java/com/codigo/msregisterhexagonal/application/ApplicationLauncher.java

Tipo: class. Rol: Clase auxiliar. Paquete: com.codigo.msregisterhexagonal.application.

- Apoya una tarea específica dentro del proyecto.
- Metodos para revisar: void main(String[] args).
- Anotaciones importantes: @ComponentScan, @EnableFeignClients, @EnableJpaRepositories, @EnableMongoRepositories, @EntityScan, @ImportAutoConfiguration, @SpringBootApplication.
- Analogía: Como una herramienta pequeña en la caja de trabajo.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

UseCaseConfig

Archivo: ms-hexagonal-reniec-openfeign-spring-boot/application/src/main/java/com/codigo/msregisterhexagonal/application/config/UseCaseConfig.java

Tipo: class. Rol: Configuración. Paquete: com.codigo.msregisterhexagonal.application.config.

- Declara beans, filtros, clientes o ajustes para que Spring los administre.
- Metodos para revisar: PersonServiceIn personServiceIn(PersonServiceOut personServiceOut).
- Anotaciones importantes: @Bean, @Configuration.
- Analogía: Como el tablero de interruptores que conecta piezas del sistema.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

ApiError

Archivo: ms-hexagonal-reniec-openfeign-spring-boot/application/src/main/java/com/codigo/msregisterhexagonal/application/controller/ApiError.java

Tipo: class. Rol: Controlador REST. Paquete: com.codigo.msregisterhexagonal.application.controller.

- Recibe solicitudes HTTP, toma parametros/body y delega al servicio.
- Sus datos/dependencias clave son: LocalDateTime timestamp, int status, String error, String message, String path.
- Anotaciones importantes: @AllArgsConstructor, @Getter.
- Analogía: Como una ventanilla: escucha al usuario y pasa el pedido a la oficina correcta.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

GlobalExceptionHandler

Archivo: ms-hexagonal-reniec-openfeign-spring-boot/application/src/main/java/com/codigo/msregisterhexagonal/application/controller/GlobalExceptionHandler.java

Tipo: class. Rol: Controlador REST. Paquete: com.codigo.msregisterhexagonal.application.controller.

- Recibe solicitudes HTTP, toma parametros/body y delega al servicio.

- Metodos para revisar: `ResponseEntity<ApiError> handlePersonNotFound(PersonNotFoundException exception, HttpServletRequest request)`, `ResponseEntity<ApiError> handleBadRequest(IllegalArgumentException exception, HttpServletRequest request)`, `ResponseEntity<ApiError> handleReniecError(ReniecException exception, HttpServletRequest request)`, `ResponseEntity<ApiError> handleUnexpectedError(Exception exception, HttpServletRequest request)`, `ResponseEntity<ApiError> buildError(HttpStatus status, String message, String path)`.
- Anotaciones importantes: `@ExceptionHandler`, `@RestControllerAdvice`.
- Analogia: Como una ventanilla: escucha al usuario y pasa el pedido a la oficina correcta.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

PersonController

Archivo: `ms-hexagonal-reniec-openfeign-spring-boot/application/src/main/java/com/codigo/msregisterhexagonal/application/controller/PersonController.java`

Tipo: class. Rol: Controlador REST. Paquete: `com.codigo.msregisterhexagonal.application.controller`.

- Recibe solicitudes HTTP, toma parametros/body y delega al servicio.
- Expone rutas HTTP como `@RequestMapping("/v1/hexagonal/")`, `@PostMapping("/save")`, `@GetMapping("/persons")`, `@GetMapping("/persons/{dni}")`.
- Sus datos/dependencias clave son: `final PersonServiceIn serviceIn`.
- Metodos para revisar: `ResponseEntity<PersonDTO> createPerson(@RequestParam("dni")`, `ResponseEntity<List<PersonDTO>> getAllPersons()`, `ResponseEntity<PersonDTO> getPersonByDni(@PathVariable String dni)`, `ResponseEntity<Void> deletePersonByDni(@PathVariable String dni)`.
- Anotaciones importantes: `@DeleteMapping`, `@GetMapping`, `@PathVariable`, `@PostMapping`, `@RequestMapping`, `@RequestParam`, `@RequiredArgsConstructor`, `@RestController`.
- Analogia: Como una ventanilla: escucha al usuario y pasa el pedido a la oficina correcta.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

PersonDTO

Archivo: `ms-hexagonal-reniec-openfeign-spring-boot/domain/src/main/java/com/codigo/msregisterhexagonal/domain/aggregates/dto/PersonDTO.java`

Tipo: class. Rol: DTO / request / response. Paquete: `com.codigo.msregisterhexagonal.domain.aggregates.dto`.

- Transporta datos entre capas o hacia/desde HTTP sin exponer directamente la entidad.
- Sus datos/dependencias clave son: `Long id`, `String numDocPerson`, `String typeDocPerson`, `String firstNamePerson`, `String lastNamePerson`, `Integer statusPerson`.
- Anotaciones importantes: `@AllArgsConstructor`, `@Getter`, `@NoArgsConstructor`, `@Setter`.
- Analogia: Como un formulario: solo lleva los datos necesarios para una operacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

PersonNotFoundException

Archivo: `ms-hexagonal-reniec-openfeign-spring-boot/domain/src/main/java/com/codigo/msregisterhexagonal/domain/exception/PersonNotFoundException.java`

Tipo: class. Rol: Excepcion / handler. Paquete: `com.codigo.msregisterhexagonal.domain.exception`.

- Representa errores o traduce excepciones en respuestas HTTP comprensibles.
- Metodos para revisar: `PersonNotFoundException(String dni)`.
- Analogia: Como un protocolo de emergencia que evita respuestas desordenadas.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

ReniecException

Archivo: `ms-hexagonal-reniec-openfeign-spring-boot/domain/src/main/java/com/codigo/msregisterhexagonal/domain/exception/ReniecException.java`

Tipo: class. Rol: Excepcion / handler. Paquete: `com.codigo.msregisterhexagonal.domain.exception`.

- Representa errores o traduce excepciones en respuestas HTTP comprensibles.
- Metodos para revisar: `ReniecException(String message)`, `ReniecException(String message, Throwable cause)`.

- Analogia: Como un protocolo de emergencia que evita respuestas desordenadas.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

PersonServiceIn

Archivo: ms-hexagonal-reniec-openfeign-spring-boot/domain/src/main/java/com/codigo/msregisterhexagonal/domain/ports/in/PersonServiceIn.java

Tipo: interface. Rol: Clase auxiliar. Paquete: com.codigo.msregisterhexagonal.domain.ports.in.

- Apoya una tarea especifica dentro del proyecto.
- Analogia: Como una herramienta pequena en la caja de trabajo.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

PersonServiceOut

Archivo: ms-hexagonal-reniec-openfeign-spring-boot/domain/src/main/java/com/codigo/msregisterhexagonal/domain/ports/out/PersonServiceOut.java

Tipo: interface. Rol: Clase auxiliar. Paquete: com.codigo.msregisterhexagonal.domain.ports.out.

- Apoya una tarea especifica dentro del proyecto.
- Analogia: Como una herramienta pequena en la caja de trabajo.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

PersonServiceImpl

Archivo: ms-hexagonal-reniec-openfeign-spring-boot/domain/src/main/java/com/codigo/msregisterhexagonal/domain/usecase/PersonServiceImpl.java

Tipo: class. Rol: Servicio de negocio. Paquete: com.codigo.msregisterhexagonal.domain.usecase.

- Implementa reglas, validaciones, llamadas a repositorios o clientes externos.
- Sus datos/dependencias clave son: final PersonServiceOut personServiceOut, String SERVICE_NAME.
- Metodos para revisar: PersonDTO createPersonIn(String dni), List<PersonDTO> getAllPersonsIn(), PersonDTO getPersonByDniIn(String dni), void deletePersonByDniIn(String dni).
- Anotaciones importantes: @Log4j2, @Override, @RequiredArgsConstructor.
- Analogia: Como la cocina: combina ingredientes y decide el proceso real.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

PersonAdapter

Archivo: ms-hexagonal-reniec-openfeign-spring-boot/infrastructure/src/main/java/com/codigo/msregisterhexagonal/infrastructure/adapters/PersonAdapter.java

Tipo: class. Rol: Clase auxiliar. Paquete: com.codigo.msregisterhexagonal.infrastructure.adapters.

- Apoya una tarea especifica dentro del proyecto.
- Sus datos/dependencias clave son: final ReniecClient reniecClient, final ModelMapper personMapper, final ModelMapper reniecMapper, final ModelMapper personDocMapper, final PersonRepository personRepository, final PersonRepositoryDoc repositoryDoc.
- Metodos para revisar: PersonDTO createPersonOut(String dni), List<PersonDTO> getAllPersonsOut(), PersonDTO getPersonByDniOut(String dni), void deletePersonByDniOut(String dni), PersonEntity getEntityForSaveSql(ResponseReniec responseReniec), PersonEntityDoc getEntityForSaveMongo(ResponseReniec responseReniec), void fillCommonFields(PersonEntity person, ResponseReniec responseReniec).
- Anotaciones importantes: @Log4j2, @Override, @Qualifier, @Service, @Value.
- Analogia: Como una herramienta pequena en la caja de trabajo.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

MapperConfig

Archivo: ms-hexagonal-reniec-openfeign-spring-boot/infrastructure/src/main/java/com/codigo/msregisterhexagonal/infrastructure/config/MapperConfig.java

Tipo: class. Rol: Configuracion. Paquete: com.codigo.msregisterhexagonal.infrastructure.config.

- Declara beans, filtros, clientes o ajustes para que Spring los administre.

- Metodos para revisar: `ModelMapper defaultMapper()`, `ModelMapper reniecMapper()`, `ModelMapper reniecMapperDoc()`.
- Anotaciones importantes: `@Bean`, `@Configuration`.
- Analogia: Como el tablero de interruptores que conecta piezas del sistema.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

PersonEntity

Archivo: `ms-hexagonal-reniec-openfeign-spring-boot/infrastructure/src/main/java/com/codigo/msregisterhexagonal/infrastructure/entity/PersonEntity.java`

Tipo: class. Rol: Entidad / modelo persistente. Paquete: `com.codigo.msregisterhexagonal.infrastructure.entity`.

- Representa datos de dominio y normalmente se mapea a tablas o documentos.
- Sus datos/dependencias clave son: Long id, String numDoc, String typeDoc, String firstName, String lastName, String motherLastName.
- Anotaciones importantes: `@Column`, `@Entity`, `@GeneratedValue`, `@Getter`, `@Id`, `@Setter`, `@Table`.
- Analogia: Como una ficha oficial con campos que describen algo del negocio.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

PersonEntityDoc

Archivo: `ms-hexagonal-reniec-openfeign-spring-boot/infrastructure/src/main/java/com/codigo/msregisterhexagonal/infrastructure/entity/PersonEntityDoc.java`

Tipo: class. Rol: Entidad / modelo persistente. Paquete: `com.codigo.msregisterhexagonal.infrastructure.entity`.

- Representa datos de dominio y normalmente se mapea a tablas o documentos.
- Sus datos/dependencias clave son: Long id, String numDoc, String typeDoc, String firstName, String lastName, String motherLastName.
- Metodos para revisar: `PersonEntityDoc()`.
- Anotaciones importantes: `@Data`, `@Document`, `@Field`.
- Analogia: Como una ficha oficial con campos que describen algo del negocio.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

PersonRepository

Archivo: `ms-hexagonal-reniec-openfeign-spring-boot/infrastructure/src/main/java/com/codigo/msregisterhexagonal/infrastructure/repository/jpa/PersonRepository.java`

Tipo: interface. Rol: Repositorio de datos. Paquete: `com.codigo.msregisterhexagonal.infrastructure.repository.jpa`.

- Abstrae la persistencia y permite consultar o guardar entidades.
- Analogia: Como el archivo o almacen donde se guarda informacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

PersonRepositoryDoc

Archivo: `ms-hexagonal-reniec-openfeign-spring-boot/infrastructure/src/main/java/com/codigo/msregisterhexagonal/infrastructure/repository/mongo/PersonRepositoryDoc.java`

Tipo: interface. Rol: Repositorio de datos. Paquete: `com.codigo.msregisterhexagonal.infrastructure.repository.mongo`.

- Abstrae la persistencia y permite consultar o guardar entidades.
- Analogia: Como el archivo o almacen donde se guarda informacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

ResponseReniec

Archivo: `ms-hexagonal-reniec-openfeign-spring-boot/infrastructure/src/main/java/com/codigo/msregisterhexagonal/infrastructure/response/ResponseReniec.java`

Tipo: class. Rol: DTO / request / response. Paquete: `com.codigo.msregisterhexagonal.infrastructure.response`.

- Transporta datos entre capas o hacia/desde HTTP sin exponer directamente la entidad.
- Sus datos/dependencias clave son: String nombres, String apellidoPaterno, String apellidoMaterno, String nombreCompleto, String tipoDocumento, String numeroDocumento.

- Anotaciones importantes: @Getter, @JsonProperty, @Setter.
- Analogia: Como un formulario: solo lleva los datos necesarios para una operacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

ReniecClient

Archivo: ms-hexagonal-reniec-openfeign-spring-boot/infrastructure/src/main/java/com/codigo/msregisterhexagonal/infrastructure/rest/ReniecClient.java

Tipo: interface. Rol: Cliente externo. Paquete: com.codigo.msregisterhexagonal.infrastructure.rest.

- Define o ejecuta llamadas HTTP hacia servicios externos.
- Expone rutas HTTP como @GetMapping("/dni").
- Anotaciones importantes: @FeignClient, @GetMapping, @RequestHeader, @RequestParam.
- Analogia: Como un mensajero que sabe la direccion y el formato del pedido.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

Preguntas para estudiar este proyecto

- Que clase recibe la solicitud inicial?
- Donde se aplica la regla de negocio?
- Que objeto representa la tabla, documento o dato principal?
- Que DTO evita exponer detalles internos?
- Que test confirma que el comportamiento se mantiene?

Hexagonal SUNAT Spring Boot

Tema principal: Arquitectura hexagonal para consulta SUNAT con dominio, puertos, adaptadores, interfaz y OpenFeign.

Analogía del proyecto: El dominio es el reglamento del negocio; los adaptadores son tramitadores que hablan con SUNAT o con la base de datos.

Dependencias y piezas visibles

Modulo Maven: hexagonal-sunat, Modulo Maven: spring-boot-starter-parent, lombok, spring-boot-starter-data-jpa, spring-boot-starter-test, spring-boot-starter-web, spring-cloud-starter-openfeign

Flujo mental

HTTP -> Controller -> Service -> Repository/Ciente externo -> Entity/DTO -> Response.

Clases y archivos explicados

ApplicationLauncher

Archivo: ms-hexagonal-sunat-openfeign-spring-boot/application/src/main/java/com/codigo/msregisterhexagonal/application/ApplicationLauncher.java

Tipo: class. Rol: Clase auxiliar. Paquete: com.codigo.msregisterhexagonal.application.

- Apoya una tarea específica dentro del proyecto.
- Metodos para revisar: void main(String[] args).
- Anotaciones importantes: @ComponentScan, @EnableFeignClients, @EnableJpaRepositories, @EntityScan, @ImportAutoConfiguration, @SpringBootApplication.
- Analogía: Como una herramienta pequeña en la caja de trabajo.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

Empresa

Archivo: ms-hexagonal-sunat-openfeign-spring-boot/domain/src/main/java/com/codigo/msregisterhexagonal/domain/model/Empresa.java

Tipo: class. Rol: Entidad / modelo persistente. Paquete: com.codigo.msregisterhexagonal.domain.model.

- Representa datos de dominio y normalmente se mapea a tablas o documentos.
- Sus datos/dependencias clave son: Long id, String razonSocial, String numeroDocumento, String estado, String condicion, String direccion.
- Anotaciones importantes: @AllArgsConstructor, @Builder, @Getter, @NoArgsConstructor, @Setter.
- Analogía: Como una ficha oficial con campos que describen algo del negocio.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

EmpresaServiceIn

Archivo: ms-hexagonal-sunat-openfeign-spring-boot/domain/src/main/java/com/codigo/msregisterhexagonal/domain/ports/in/EmpresaServiceIn.java

Tipo: interface. Rol: Clase auxiliar. Paquete: com.codigo.msregisterhexagonal.domain.ports.in.

- Apoya una tarea específica dentro del proyecto.
- Analogía: Como una herramienta pequeña en la caja de trabajo.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

EmpresaServiceOut

Archivo: ms-hexagonal-sunat-openfeign-spring-boot/domain/src/main/java/com/codigo/msregisterhexagonal/domain/ports/out/EmpresaServiceOut.java

Tipo: interface. Rol: Clase auxiliar. Paquete: com.codigo.msregisterhexagonal.domain.ports.out.

- Apoya una tarea específica dentro del proyecto.
- Analogía: Como una herramienta pequeña en la caja de trabajo.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

SunatServiceOut

Archivo: ms-hexagonal-sunat-openfeign-spring-boot/domain/src/main/java/com/codigo/msregisterhexagonal/domain/ports/out/SunatServiceOut.java

Tipo: interface. Rol: Clase auxiliar. Paquete: com.codigo.msregisterhexagonal.domain.ports.out.

- Apoya una tarea especifica dentro del proyecto.
- Analogia: Como una herramienta pequena en la caja de trabajo.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

EmpresaServiceImpl

Archivo: ms-hexagonal-sunat-openfeign-spring-boot/domain/src/main/java/com/codigo/msregisterhexagonal/domain/usecase/EmpresaServiceImpl.java

Tipo: class. Rol: Servicio de negocio. Paquete: com.codigo.msregisterhexagonal.domain.usecase.

- Implementa reglas, validaciones, llamadas a repositorios o clientes externos.
- Sus datos/dependencias clave son: final EmpresaServiceOut empresaServiceOut, final SunatServiceOut sunatServiceOut.
- Metodos para revisar: Empresa guardar(Empresa empresa), Empresa guardarDesdeSunat(String ruc), Optional<Empresa> buscarPorId(Long id), List<Empresa> listar(), Empresa actualizar(Long id, Empresa empresa), void eliminar(Long id), EmpresaServiceImpl(EmpresaServiceOut empresaServiceOut, SunatServiceOut sunatServiceOut).
- Anotaciones importantes: @Override.
- Analogia: Como la cocina: combina ingredientes y decide el proceso real.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

SunatClientAdapter

Archivo: ms-hexagonal-sunat-openfeign-spring-boot/infrastructure/src/main/java/com/codigo/msregisterhexagonal/infrastructure/client/SunatClientAdapter.java

Tipo: class. Rol: Cliente externo. Paquete: com.codigo.msregisterhexagonal.infrastructure.client.

- Define o ejecuta llamadas HTTP hacia servicios externos.
- Sus datos/dependencias clave son: final SunatFeignClient sunatFeignClient.
- Metodos para revisar: Empresa consultarPorRuc(String ruc), Empresa toEmpresa(SunatResponse sunatResponse), SunatClientAdapter(SunatFeignClient sunatFeignClient).
- Anotaciones importantes: @Component, @Override.
- Analogia: Como un mensajero que sabe la direccion y el formato del pedido.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

SunatFeignClient

Archivo: ms-hexagonal-sunat-openfeign-spring-boot/infrastructure/src/main/java/com/codigo/msregisterhexagonal/infrastructure/client/SunatFeignClient.java

Tipo: interface. Rol: Cliente externo. Paquete: com.codigo.msregisterhexagonal.infrastructure.client.

- Define o ejecuta llamadas HTTP hacia servicios externos.
- Expone rutas HTTP como @GetMapping("ruc").
- Anotaciones importantes: @FeignClient, @GetMapping, @RequestParam.
- Analogia: Como un mensajero que sabe la direccion y el formato del pedido.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

SunatResponse

Archivo: ms-hexagonal-sunat-openfeign-spring-boot/infrastructure/src/main/java/com/codigo/msregisterhexagonal/infrastructure/client/SunatResponse.java

Tipo: class. Rol: Cliente externo. Paquete: com.codigo.msregisterhexagonal.infrastructure.client.

- Define o ejecuta llamadas HTTP hacia servicios externos.
- Sus datos/dependencias clave son: String ruc, String razonSocial, String numeroDocumento, String nombreComercial, List<String> telefonos, String tipo.

- Anotaciones importantes: @AllArgsConstructor, @Getter, @JsonIgnoreProperties, @JsonProperty, @NoArgsConstructor, @Setter.
- Analogia: Como un mensajero que sabe la direccion y el formato del pedido.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

DecolectaProperties

Archivo: ms-hexagonal-sunat-openfeign-spring-boot/infrastructure/src/main/java/com/codigo/msregisterhexagonal/infrastructure/config/DecolectaProperties.java

Tipo: class. Rol: Configuracion. Paquete: com.codigo.msregisterhexagonal.infrastructure.config.

- Declara beans, filtros, clientes o ajustes para que Spring los administre.
- Sus datos/dependencias clave son: String baseUrl, String token.
- Metodos para revisar: String getBaseUrl(), void setBaseUrl(String baseUrl), String getToken(), void setToken(String token).
- Anotaciones importantes: @Component, @ConfigurationProperties.
- Analogia: Como el tablero de interruptores que conecta piezas del sistema.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

FeignConfig

Archivo: ms-hexagonal-sunat-openfeign-spring-boot/infrastructure/src/main/java/com/codigo/msregisterhexagonal/infrastructure/config/FeignConfig.java

Tipo: class. Rol: Configuracion. Paquete: com.codigo.msregisterhexagonal.infrastructure.config.

- Declara beans, filtros, clientes o ajustes para que Spring los administre.
- Metodos para revisar: RequestInterceptor decolectaRequestInterceptor(DecolectaProperties decolectaProperties).
- Anotaciones importantes: @Bean, @Configuration.
- Analogia: Como el tablero de interruptores que conecta piezas del sistema.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

UseCaseConfig

Archivo: ms-hexagonal-sunat-openfeign-spring-boot/infrastructure/src/main/java/com/codigo/msregisterhexagonal/infrastructure/config/UseCaseConfig.java

Tipo: class. Rol: Configuracion. Paquete: com.codigo.msregisterhexagonal.infrastructure.config.

- Declara beans, filtros, clientes o ajustes para que Spring los administre.
- Metodos para revisar: EmpresaServiceIn empresaServiceIn(EmpresaServiceOut empresaServiceOut, SunatServiceOut sunatServiceOut).
- Anotaciones importantes: @Bean, @Configuration.
- Analogia: Como el tablero de interruptores que conecta piezas del sistema.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

EmpresaEntity

Archivo: ms-hexagonal-sunat-openfeign-spring-boot/infrastructure/src/main/java/com/codigo/msregisterhexagonal/infrastructure/entity/EmpresaEntity.java

Tipo: class. Rol: Entidad / modelo persistente. Paquete: com.codigo.msregisterhexagonal.infrastructure.entity.

- Representa datos de dominio y normalmente se mapea a tablas o documentos.
- Sus datos/dependencias clave son: Long id, String razonSocial, String numeroDocumento, String estado, String condicion, String direccion.
- Anotaciones importantes: @AllArgsConstructor, @Builder, @Column, @Entity, @GeneratedValue, @Getter, @Id, @NoArgsConstructor.
- Analogia: Como una ficha oficial con campos que describen algo del negocio.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

ConsultaSunatException

Archivo: ms-hexagonal-sunat-openfeign-spring-boot/infrastructure/src/main/java/com/codigo/msregisterhexagonal/infrastructure/exception/ConsultaSunatException.java

Tipo: class. Rol: Excepcion / handler. Paquete: com.codigo.msregisterhexagonal.infrastructure.exception.

- Representa errores o traduce excepciones en respuestas HTTP comprensibles.
- Metodos para revisar: ConsultaSunatException(String message), ConsultaSunatException(String message, Throwable cause).
- Analogia: Como un protocolo de emergencia que evita respuestas desordenadas.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

EmpresaRepository

Archivo: ms-hexagonal-sunat-openfeign-spring-boot/infrastructure/src/main/java/com/codigo/msregisterhexagonal/infrastructure/repository/EmpresaRepository.java

Tipo: interface. Rol: Repositorio de datos. Paquete: com.codigo.msregisterhexagonal.infrastructure.repository.

- Abstrae la persistencia y permite consultar o guardar entidades.
- Analogia: Como el archivo o almacen donde se guarda informacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

EmpresaRepositoryAdapter

Archivo: ms-hexagonal-sunat-openfeign-spring-boot/infrastructure/src/main/java/com/codigo/msregisterhexagonal/infrastructure/repository/EmpresaRepositoryAdapter.java

Tipo: class. Rol: Repositorio de datos. Paquete: com.codigo.msregisterhexagonal.infrastructure.repository.

- Abstrae la persistencia y permite consultar o guardar entidades.
- Sus datos/dependencias clave son: final EmpresaRepository empresaRepository.
- Metodos para revisar: Empresa guardar(Empresa empresa), Optional<Empresa> buscarPorId(Long id), List<Empresa> listar(), Empresa actualizar(Long id, Empresa empresa), void eliminar(Long id), EmpresaEntity toEntity(Empresa empresa), Empresa toDomain(EmpresaEntity empresaEntity).
- Anotaciones importantes: @Override, @Repository.
- Analogia: Como el archivo o almacen donde se guarda informacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

EmpresaController

Archivo: ms-hexagonal-sunat-openfeign-spring-boot/interface/src/main/java/com/codigo/msregisterhexagonal/interfaces/controller/EmpresaController.java

Tipo: class. Rol: Controlador REST. Paquete: com.codigo.msregisterhexagonal.interfaces.controller.

- Recibe solicitudes HTTP, toma parametros/body y delega al servicio.
- Expone rutas HTTP como @RequestMapping("/api/empresas"), @PostMapping, @PostMapping("/sunat/{ruc}"), @GetMapping("/{id}").
- Sus datos/dependencias clave son: final EmpresaServiceIn empresaServiceIn.
- Metodos para revisar: ResponseBase<Empresa> guardar(@RequestBody Empresa empresa), ResponseBase<Empresa> guardarDesdeSunat(@PathVariable String ruc), ResponseBase<Empresa> buscarPorId(@PathVariable Long id), ResponseBase<List<Empresa>> listar(), ResponseBase<Empresa> actualizar(@PathVariable Long id, @RequestBody Empresa empresa), ResponseBase<Object> eliminar(@PathVariable Long id), EmpresaController(EmpresaServiceIn empresaServiceIn).
- Anotaciones importantes: @DeleteMapping, @GetMapping, @PathVariable, @PostMapping, @PutMapping, @RequestBody, @RequestMapping, @ResponseStatus.
- Analogia: Como una ventanilla: escucha al usuario y pasa el pedido a la oficina correcta.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

ResponseBase

Archivo: ms-hexagonal-sunat-openfeign-spring-boot/interface/src/main/java/com/codigo/msregisterhexagonal/interfaces/dto/ResponseBase.java

Tipo: class. Rol: DTO / request / response. Paquete: com.codigo.msregisterhexagonal.interfaces.dto.

- Transporta datos entre capas o hacia/desde HTTP sin exponer directamente la entidad.
- Sus datos/dependencias clave son: Integer codigo, String mensaje, T data.
- Metodos para revisar: `<T> ResponseBase<T> success(Integer codigo, String mensaje, T data)`, `ResponseBase<Object> error(Integer codigo, String mensaje)`.
- Anotaciones importantes: `@AllArgsConstructor`, `@Getter`, `@NoArgsConstructor`, `@Setter`.
- Analogia: Como un formulario: solo lleva los datos necesarios para una operacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

GlobalExceptionHandler

Archivo: `ms-hexagonal-sunat-openfeign-spring-boot/interface/src/main/java/com/codigo/msregisterhexagonal/interfaces/exception/GlobalExceptionHandler.java`

Tipo: class. Rol: Excepcion / handler. Paquete: `com.codigo.msregisterhexagonal.interfaces.exception`.

- Representa errores o traduce excepciones en respuestas HTTP comprensibles.
- Metodos para revisar: `ResponseEntity<ResponseBase<Object>> handleBadRequest(Exception exception)`, `ResponseEntity<ResponseBase<Object>> handleNotFound(NoSuchElementException exception)`, `ResponseEntity<ResponseBase<Object>> handleInternalServerError(Exception exception)`.
- Anotaciones importantes: `@ExceptionHandler`, `@RestControllerAdvice`.
- Analogia: Como un protocolo de emergencia que evita respuestas desordenadas.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

Preguntas para estudiar este proyecto

- Que clase recibe la solicitud inicial?
- Donde se aplica la regla de negocio?
- Que objeto representa la tabla, documento o dato principal?
- Que DTO evita exponer detalles internos?
- Que test confirma que el comportamiento se mantiene?

Patrones Strategy, Adapter y Builder

Tema principal: Tres proyectos para practicar patrones de diseño aplicados con Spring Boot.

Analogía del proyecto: Strategy cambia la herramienta según el caso, Adapter conecta enchufes incompatibles y Builder arma objetos paso a paso.

Dependencias y piezas visibles

Módulo Maven: spring-boot-starter-parent, lombok, spring-boot-starter-test, spring-boot-starter-web

Flujo mental

Clases conectadas por Spring Boot según rol: configuración, servicio, modelo y prueba.

Clases y archivos explicados

AdapterBiblioteca

Archivo: patrones-strategy-adapter-builder-spring-boot/patron-adapter/src/main/java/com/codigo/patron_adapter/adapter/AdapterBiblioteca.java

Tipo: class. Rol: Adapter. Paquete: com.codigo.patron_adapter.adapter.

- Adapta una interfaz antigua o externa a una forma que el sistema entiende.
- Sus datos/dependencias clave son: final Biblioteca biblioteca, final XmlMapper xmlMapper.
- Métodos para revisar: String obtenerDetalle(int idLibro), Libro obtenerDetalle2(int idLibro).
- Anotaciones importantes: @Component.
- Analogía: Como un adaptador de enchufe.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

Biblioteca

Archivo: patrones-strategy-adapter-builder-spring-boot/patron-adapter/src/main/java/com/codigo/patron_adapter/antiguo/Biblioteca.java

Tipo: class. Rol: Clase auxiliar. Paquete: com.codigo.patron_adapter.antiguo.

- Apoya una tarea específica dentro del proyecto.
- Métodos para revisar: String obtenerInfo(int idLibro), String obtenerInfo2(int idLibro).
- Anotaciones importantes: @example.
- Analogía: Como una herramienta pequeña en la caja de trabajo.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

BibliotecaController

Archivo: patrones-strategy-adapter-builder-spring-boot/patron-adapter/src/main/java/com/codigo/patron_adapter/controller/BibliotecaController.java

Tipo: class. Rol: Controlador REST. Paquete: com.codigo.patron_adapter.controller.

- Recibe solicitudes HTTP, toma parámetros/body y delega al servicio.
- Expone rutas HTTP como @RequestMapping("/v1/adapter"), @GetMapping(value, @GetMapping(value).
- Sus datos/dependencias clave son: final AdapterBiblioteca biblioteca.
- Métodos para revisar: ResponseEntity<String> obtenerLibro(@PathVariable Integer id), ResponseEntity<Libro> obtenerLibro2(@PathVariable Integer id), BibliotecaController(AdapterBiblioteca biblioteca).
- Anotaciones importantes: @GetMapping, @PathVariable, @RequestMapping, @RestController.
- Analogía: Como una ventanilla: escucha al usuario y pasa el pedido a la oficina correcta.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

Autor

Archivo: patrones-strategy-adapter-builder-spring-boot/patron-adapter/src/main/java/com/codigo/patron_adapter/model/Autor.java

Tipo: class. Rol: Entidad / modelo persistente. Paquete: com.codigo.patron_adapter.model.

- Representa datos de dominio y normalmente se mapea a tablas o documentos.
- Sus datos/dependencias clave son: String nombre, String nacionalidad.
- Anotaciones importantes: @Getter, @NoArgsConstructor, @Setter.
- Analogia: Como una ficha oficial con campos que describen algo del negocio.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

Contacto

Archivo: patrones-strategy-adapter-builder-spring-boot/patron-adapter/src/main/java/com/codigo/patron_adapter/model/Contacto.java

Tipo: class. Rol: Entidad / modelo persistente. Paquete: com.codigo.patron_adapter.model.

- Representa datos de dominio y normalmente se mapea a tablas o documentos.
- Sus datos/dependencias clave son: String email, String telefono.
- Anotaciones importantes: @Getter, @NoArgsConstructor, @Setter.
- Analogia: Como una ficha oficial con campos que describen algo del negocio.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

Editorial

Archivo: patrones-strategy-adapter-builder-spring-boot/patron-adapter/src/main/java/com/codigo/patron_adapter/model/Editorial.java

Tipo: class. Rol: Entidad / modelo persistente. Paquete: com.codigo.patron_adapter.model.

- Representa datos de dominio y normalmente se mapea a tablas o documentos.
- Sus datos/dependencias clave son: String nombre, String direccion, String ciudad, Contacto contacto.
- Anotaciones importantes: @Getter, @NoArgsConstructor, @Setter.
- Analogia: Como una ficha oficial con campos que describen algo del negocio.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

Libro

Archivo: patrones-strategy-adapter-builder-spring-boot/patron-adapter/src/main/java/com/codigo/patron_adapter/model/Libro.java

Tipo: class. Rol: Entidad / modelo persistente. Paquete: com.codigo.patron_adapter.model.

- Representa datos de dominio y normalmente se mapea a tablas o documentos.
- Sus datos/dependencias clave son: int id, String titulo, String genero, int anio, Editorial editorial, List<Autor> autores.
- Anotaciones importantes: @Getter, @NoArgsConstructor, @Setter.
- Analogia: Como una ficha oficial con campos que describen algo del negocio.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

Ubicacion

Archivo: patrones-strategy-adapter-builder-spring-boot/patron-adapter/src/main/java/com/codigo/patron_adapter/model/Ubicacion.java

Tipo: class. Rol: Entidad / modelo persistente. Paquete: com.codigo.patron_adapter.model.

- Representa datos de dominio y normalmente se mapea a tablas o documentos.
- Sus datos/dependencias clave son: int piso, String estanteria, String sucursal.
- Anotaciones importantes: @Getter, @NoArgsConstructor, @Setter.
- Analogia: Como una ficha oficial con campos que describen algo del negocio.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

PatronAdapterApplication

Archivo: patrones-strategy-adapter-builder-spring-boot/patron-adapter/src/main/java/com/codigo/patron_adapter/PatronAdapterApplication.java

Tipo: class. Rol: Punto de arranque. Paquete: com.codigo.patron_adapter.

- Enciende Spring Boot, escanea componentes y levanta el servidor embebido.
- Metodos para revisar: void main(String[] args).
- Anotaciones importantes: @SpringBootApplication.
- Analogia: Como girar la llave de contacto de un auto.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

PatronAdapterApplicationTests

Archivo: patrones-strategy-adapter-builder-spring-boot/patron-adapter/src/test/java/com/codigo/patron_adapter/PatronAdapterApplicationTests.java

Tipo: class. Rol: Prueba. Paquete: com.codigo.patron_adapter.

- Verifica comportamiento de una clase, endpoint o contexto.
- Anotaciones importantes: @SpringBootTest, @Test.
- Analogia: Como una lista de chequeo antes de despegar.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

CarroController

Archivo: patrones-strategy-adapter-builder-spring-boot/patron-builder/src/main/java/com/codigo/patron_builder/controller/CarroController.java

Tipo: class. Rol: Controlador REST. Paquete: com.codigo.patron_builder.controller.

- Recibe solicitudes HTTP, toma parametros/body y delega al servicio.
- Expone rutas HTTP como @RequestMapping("/v1/builder"), @GetMapping("/carrofull").
- Metodos para revisar: ResponseEntity<Carro> obtenerCarro().
- Anotaciones importantes: @GetMapping, @RequestMapping, @RestController.
- Analogia: Como una ventanilla: escucha al usuario y pasa el pedido a la oficina correcta.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

Carro

Archivo: patrones-strategy-adapter-builder-spring-boot/patron-builder/src/main/java/com/codigo/patron_builder/model/Carro.java

Tipo: class. Rol: Entidad / modelo persistente. Paquete: com.codigo.patron_builder.model.

- Representa datos de dominio y normalmente se mapea a tablas o documentos.
- Sus datos/dependencias clave son: Long id, String modelo, Integer anio, String marca, TipoCarro tipoCarro.
- Anotaciones importantes: @AllArgsConstructor, @Builder, @Getter, @NoArgsConstructor, @Setter.
- Analogia: Como una ficha oficial con campos que describen algo del negocio.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

TipoCarro

Archivo: patrones-strategy-adapter-builder-spring-boot/patron-builder/src/main/java/com/codigo/patron_builder/model/TipoCarro.java

Tipo: class. Rol: Entidad / modelo persistente. Paquete: com.codigo.patron_builder.model.

- Representa datos de dominio y normalmente se mapea a tablas o documentos.
- Sus datos/dependencias clave son: Long id, String tipo.
- Anotaciones importantes: @AllArgsConstructor, @Builder, @Getter, @NoArgsConstructor, @Setter.
- Analogia: Como una ficha oficial con campos que describen algo del negocio.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

PatronBuilderApplication

Archivo: patrones-strategy-adapter-builder-spring-boot/patron-builder/src/main/java/com/codigo/patron_builder/PatronBuilderApplication.java

Tipo: class. Rol: Punto de arranque. Paquete: com.codigo.patron_builder.

- Enciende Spring Boot, escanea componentes y levanta el servidor embebido.
- Metodos para revisar: void main(String[] args).
- Anotaciones importantes: @SpringBootApplication.
- Analogia: Como girar la llave de contacto de un auto.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

PatronBuilderApplicationTests

Archivo: patrones-strategy-adapter-builder-spring-boot/patron-builder/src/test/java/com/codigo/patron_builder/PatronBuilderApplicationTests.java

Tipo: class. Rol: Prueba. Paquete: com.codigo.patron_builder.

- Verifica comportamiento de una clase, endpoint o contexto.
- Anotaciones importantes: @SpringBootTest, @Test.
- Analogia: Como una lista de chequeo antes de despegar.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

Constants

Archivo: patrones-strategy-adapter-builder-spring-boot/patron-strategy/src/main/java/com/codigo/patron_strategy/constant/Constants.java

Tipo: class. Rol: Constantes. Paquete: com.codigo.patron_strategy.constant.

- Centraliza textos, codigos o valores compartidos.
- Sus datos/dependencias clave son: String PAGANDO.
- Analogia: Como una libreta de etiquetas oficiales para no escribir lo mismo a mano.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

PaymentController

Archivo: patrones-strategy-adapter-builder-spring-boot/patron-strategy/src/main/java/com/codigo/patron_strategy/controller/PaymentController.java

Tipo: class. Rol: Controlador REST. Paquete: com.codigo.patron_strategy.controller.

- Recibe solicitudes HTTP, toma parametros/body y delega al servicio.
- Expone rutas HTTP como @RequestMapping("/v1/strategy"), @PostMapping("/credit-card"), @PostMapping("/debit-card"), @PostMapping("/crypto").
- Sus datos/dependencias clave son: final PaymentService paymentService.
- Metodos para revisar: ResponseEntity<String> payWithCreditCard(@RequestParam double amount, @RequestParam String cardNumber, @RequestParam String cardHolderName, @RequestParam String numShare), ResponseEntity<String> payWithDebitCard(@RequestParam double amount, @RequestParam String cardNumber, @RequestParam String cardHolderName), ResponseEntity<String> payWithCrypto(@RequestParam double amount, @RequestParam String wallet), ResponseEntity<String> payWithPlin(@RequestParam double amount, @RequestParam String numberCelphone, @RequestParam String name), PaymentController(PaymentService paymentService).
- Anotaciones importantes: @PostMapping, @RequestMapping, @RequestParam, @RestController.
- Analogia: Como una ventanilla: escucha al usuario y pasa el pedido a la oficina correcta.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

PatronStrategyApplication

Archivo: patrones-strategy-adapter-builder-spring-boot/patron-strategy/src/main/java/com/codigo/patron_strategy/PatronStrategyApplication.java

Tipo: class. Rol: Punto de arranque. Paquete: com.codigo.patron_strategy.

- Enciende Spring Boot, escanea componentes y levanta el servidor embebido.
- Metodos para revisar: void main(String[] args).
- Anotaciones importantes: @SpringBootApplication.

- Analogia: Como girar la llave de contacto de un auto.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

PaymentService

Archivo: patrones-strategy-adapter-builder-spring-boot/patron-strategy/src/main/java/com/codigo/patron_strategy/service/PaymentService.java

Tipo: class. Rol: Clase auxiliar. Paquete: com.codigo.patron_strategy.service.

- Apoya una tarea especifica dentro del proyecto.
- Sus datos/dependencias clave son: PaymentStrategy paymentStrategy.
- Metodos para revisar: void setPaymentStrategy(PaymentStrategy paymentStrategy), void processPayment(double amount).
- Anotaciones importantes: @Log4j2, @Service.
- Analogia: Como una herramienta pequena en la caja de trabajo.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

CreditCardPayment

Archivo: patrones-strategy-adapter-builder-spring-boot/patron-strategy/src/main/java/com/codigo/patron_strategy/strategy/impl/CreditCardPayment.java

Tipo: class. Rol: Strategy. Paquete: com.codigo.patron_strategy.strategy.impl.

- Permite intercambiar algoritmos que comparten una misma interfaz.
- Sus datos/dependencias clave son: String cardNumber, String cardHolderName, String numShare.
- Metodos para revisar: void pay(double amount).
- Anotaciones importantes: @AllArgsConstructor, @Log4j2, @Override.
- Analogia: Como escoger metodo de pago sin cambiar la caja.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

CryptoPayment

Archivo: patrones-strategy-adapter-builder-spring-boot/patron-strategy/src/main/java/com/codigo/patron_strategy/strategy/impl/CryptoPayment.java

Tipo: class. Rol: Strategy. Paquete: com.codigo.patron_strategy.strategy.impl.

- Permite intercambiar algoritmos que comparten una misma interfaz.
- Sus datos/dependencias clave son: String walletAddress.
- Metodos para revisar: void pay(double amount).
- Anotaciones importantes: @AllArgsConstructor, @Log4j2, @Override.
- Analogia: Como escoger metodo de pago sin cambiar la caja.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

DebitCardPayment

Archivo: patrones-strategy-adapter-builder-spring-boot/patron-strategy/src/main/java/com/codigo/patron_strategy/strategy/impl/DebitCardPayment.java

Tipo: class. Rol: Strategy. Paquete: com.codigo.patron_strategy.strategy.impl.

- Permite intercambiar algoritmos que comparten una misma interfaz.
- Sus datos/dependencias clave son: String cardNumber, String cardHolderName.
- Metodos para revisar: void pay(double amount).
- Anotaciones importantes: @AllArgsConstructor, @Log4j2, @Override.
- Analogia: Como escoger metodo de pago sin cambiar la caja.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

PlinPayment

Archivo: patrones-strategy-adapter-builder-spring-boot/patron-strategy/src/main/java/com/codigo/patron_strategy/strategy/impl/PlinPayment.java

Tipo: class. Rol: Strategy. Paquete: com.codigo.patron_strategy.strategy.impl.

- Permite intercambiar algoritmos que comparten una misma interfaz.
- Sus datos/dependencias clave son: String numberCelphone, String name.
- Metodos para revisar: void pay(double amount).
- Anotaciones importantes: @AllArgsConstructor, @Log4j2, @Override.
- Analogia: Como escoger metodo de pago sin cambiar la caja.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

PaymentStrategy

Archivo: patrones-strategy-adapter-builder-spring-boot/patron-strategy/src/main/java/com/codigo/patron_strategy/strategy/PaymentStrategy.java

Tipo: interface. Rol: Strategy. Paquete: com.codigo.patron_strategy.strategy.

- Permite intercambiar algoritmos que comparten una misma interfaz.
- Analogia: Como escoger metodo de pago sin cambiar la caja.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

PatronStrategyApplicationTests

Archivo: patrones-strategy-adapter-builder-spring-boot/patron-strategy/src/test/java/com/codigo/patron_strategy/PatronStrategyApplicationTests.java

Tipo: class. Rol: Prueba. Paquete: com.codigo.patron_strategy.

- Verifica comportamiento de una clase, endpoint o contexto.
- Anotaciones importantes: @SpringBootTest, @Test.
- Analogia: Como una lista de chequeo antes de despegar.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

Preguntas para estudiar este proyecto

- Que clase recibe la solicitud inicial?
- Donde se aplica la regla de negocio?
- Que objeto representa la tabla, documento o dato principal?
- Que DTO evita exponer detalles internos?
- Que test confirma que el comportamiento se mantiene?

Unit Testing Empresa Spring Boot

Tema principal: Pruebas unitarias, JUnit, Mockito, MockMvc, JaCoCo, CI y buenas practicas.

Analogia del proyecto: Las pruebas son el laboratorio del proyecto: verifican piezas aisladas antes de confiar en todo el sistema.

Dependencias y piezas visibles

Modulo Maven: spring-boot-starter-parent, lombok, spring-boot-starter-data-jpa, spring-boot-starter-test, spring-boot-starter-web

Flujo mental

HTTP -> Controller -> Service -> Repository/Ciente externo -> Entity/DTO -> Response.

Clases y archivos explicados

Constants

Archivo:
unit-testing-empresa-api-spring-boot/src/main/java/com/codigo/unit_testing/aggregates/constants/Constants.java

Tipo: class. Rol: DTO / request / response. Paquete: com.codigo.unit_testing.aggregates.constants.

- Transporta datos entre capas o hacia/desde HTTP sin exponer directamente la entidad.
- Sus datos/dependencias clave son: Integer STATUS_ACTIVE, String AUDIT_ADMIN, String CONDICION, Integer CODE_OK, String MSJ_OK, Integer CODE_EXIST.
- Metodos para revisar: Constants().
- Analogia: Como un formulario: solo lleva los datos necesarios para una operacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

EmpresaRequest

Archivo: unit-testing-empresa-api-spring-boot/src/main/java/com/codigo/unit_testing/aggregates/request/EmpresaRequest.java

Tipo: class. Rol: DTO / request / response. Paquete: com.codigo.unit_testing.aggregates.request.

- Transporta datos entre capas o hacia/desde HTTP sin exponer directamente la entidad.
- Sus datos/dependencias clave son: String razonSocial, String tipoDocumento, String numeroDocumento, String direccion, String distrito, String provincia.
- Anotaciones importantes: @Getter, @Setter.
- Analogia: Como un formulario: solo lleva los datos necesarios para una operacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

BaseResponse

Archivo: unit-testing-empresa-api-spring-boot/src/main/java/com/codigo/unit_testing/aggregates/response/BaseResponse.java

Tipo: class. Rol: DTO / request / response. Paquete: com.codigo.unit_testing.aggregates.response.

- Transporta datos entre capas o hacia/desde HTTP sin exponer directamente la entidad.
- Sus datos/dependencias clave son: Integer code, String message, Optional<T> objeto.
- Anotaciones importantes: @AllArgsConstructor, @Getter, @NoArgsConstructor, @Setter.
- Analogia: Como un formulario: solo lleva los datos necesarios para una operacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

EmpresaController

Archivo:
unit-testing-empresa-api-spring-boot/src/main/java/com/codigo/unit_testing/controller/EmpresaController.java

Tipo: class. Rol: Controlador REST. Paquete: com.codigo.unit_testing.controller.

- Recibe solicitudes HTTP, toma parametros/body y delega al servicio.

- Expone rutas HTTP como `@RequestMapping("/empresa/v1/")`, `@PostMapping`, `@GetMapping("/{id}")`, `@GetMapping()`.
- Sus datos/dependencias clave son: `final EmpresaService service`.
- Metodos para revisar: `ResponseEntity<BaseResponse<Empresa>> registrar(@RequestBody EmpresaRequest empresaRequest)`.
- Anotaciones importantes: `@GetMapping`, `@PathVariable`, `@PostMapping`, `@PutMapping`, `@RequestBody`, `@RequestMapping`, `@RequiredArgsConstructor`, `@RestController`.
- Analogia: Como una ventanilla: escucha al usuario y pasa el pedido a la oficina correcta.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

EmpresaRepository

Archivo: `unit-testing-empresa-api-spring-boot/src/main/java/com/codigo/unit_testing/dao/EmpresaRepository.java`

Tipo: interface. Rol: Repositorio de datos. Paquete: `com.codigo.unit_testing.dao`.

- Abstrae la persistencia y permite consultar o guardar entidades.
- Analogia: Como el archivo o almacen donde se guarda informacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

Empresa

Archivo: `unit-testing-empresa-api-spring-boot/src/main/java/com/codigo/unit_testing/entity/Empresa.java`

Tipo: class. Rol: Entidad / modelo persistente. Paquete: `com.codigo.unit_testing.entity`.

- Representa datos de dominio y normalmente se mapea a tablas o documentos.
- Sus datos/dependencias clave son: `Long id`, `String razonSocial`, `String tipoDocumento`, `String numeroDocumento`, `String condicion`, `String direccion`.
- Anotaciones importantes: `@Entity`, `@GeneratedValue`, `@Getter`, `@Id`, `@Setter`, `@Table`.
- Analogia: Como una ficha oficial con campos que describen algo del negocio.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

EmpresaService

Archivo: `unit-testing-empresa-api-spring-boot/src/main/java/com/codigo/unit_testing/service/EmpresaService.java`

Tipo: interface. Rol: Contrato de servicio. Paquete: `com.codigo.unit_testing.service`.

- Define casos de uso disponibles sin revelar la implementacion.
- Analogia: Como el menu de un restaurante: dice que platos existen, no como se cocinan.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

EmpresaServiceImpl

Archivo:
`unit-testing-empresa-api-spring-boot/src/main/java/com/codigo/unit_testing/service/impl/EmpresaServiceImpl.java`

Tipo: class. Rol: Servicio de negocio. Paquete: `com.codigo.unit_testing.service.impl`.

- Implementa reglas, validaciones, llamadas a repositorios o clientes externos.
- Sus datos/dependencias clave son: `final EmpresaRepository empresaRepository`.
- Metodos para revisar: `ResponseEntity<BaseResponse<Empresa>> crear(EmpresaRequest request)`, `ResponseEntity<BaseResponse<Empresa>> obtenerEmpresa(Long id)`, `ResponseEntity<BaseResponse<List<Empresa>>> obtenerTodos()`, `ResponseEntity<BaseResponse<Empresa>> actualizar(Long id, EmpresaRequest empresaRequest)`, `ResponseEntity<BaseResponse<Empresa>> delete(Long id)`, `ResponseEntity<BaseResponse<Empresa>> obtenerEmpresaXNumDoc(String numDocu)`, `Empresa toEntity(EmpresaRequest request)`.
- Anotaciones importantes: `@Override`, `@RequiredArgsConstructor`, `@Service`.
- Analogia: Como la cocina: combina ingredientes y decide el proceso real.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

UnitTestingApplication

Archivo: `unit-testing-empresa-api-spring-boot/src/main/java/com/codigo/unit_testing/UnitTestingApplication.java`

Tipo: class. Rol: Punto de arranque. Paquete: com.codigo.unit_testing.

- Enciende Spring Boot, escanea componentes y levanta el servidor embebido.
- Metodos para revisar: void main(String[] args).
- Anotaciones importantes: @SpringBootApplication.
- Analogia: Como girar la llave de contacto de un auto.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

EmpresaControllerTest

Archivo: unit-testing-empresa-api-spring-boot/src/test/java/com/codigo/unit_testing/controller/EmpresaControllerTest.java

Tipo: class. Rol: Prueba. Paquete: com.codigo.unit_testing.controller.

- Verifica comportamiento de una clase, endpoint o contexto.
- Sus datos/dependencias clave son: MockMvc mockMvc, EmpresaService empresaService, ObjectMapper objectMapper.
- Anotaciones importantes: @Autowired, @DisplayName, @MockBean, @Test, @WebMvcTest.
- Analogia: Como una lista de chequeo antes de despegar.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

EmpresaServiceImplTest

Archivo: unit-testing-empresa-api-spring-boot/src/test/java/com/codigo/unit_testing/service/impl/EmpresaServiceImplTest.java

Tipo: class. Rol: Prueba. Paquete: com.codigo.unit_testing.service.impl.

- Verifica comportamiento de una clase, endpoint o contexto.
- Sus datos/dependencias clave son: EmpresaRepository empresaRepository, EmpresaServiceImpl empresaService, Empresa empresa, EmpresaRequest empresaRequest.
- Anotaciones importantes: @BeforeEach, @DisplayName, @InjectMocks, @Mock, @Test.
- Analogia: Como una lista de chequeo antes de despegar.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

UnitTestingApplicationTests

Archivo:
unit-testing-empresa-api-spring-boot/src/test/java/com/codigo/unit_testing/UnitTestingApplicationTests.java

Tipo: class. Rol: Prueba. Paquete: com.codigo.unit_testing.

- Verifica comportamiento de una clase, endpoint o contexto.
- Anotaciones importantes: @SpringBootTest, @Test.
- Analogia: Como una lista de chequeo antes de despegar.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

Preguntas para estudiar este proyecto

- Que clase recibe la solicitud inicial?
- Donde se aplica la regla de negocio?
- Que objeto representa la tabla, documento o dato principal?
- Que DTO evita exponer detalles internos?
- Que test confirma que el comportamiento se mantiene?

Repositorio de configuracion

Tema principal: Configuracion centralizada para microservicios con Spring Cloud Config.

Analogia del proyecto: Es el tablero electrico de una casa: desde un punto se distribuyen propiedades a varios servicios.

Flujo mental

main.jsx -> rutas -> layout -> paginas -> componentes -> hook/servicio -> estado del navegador.

Clases y archivos explicados

Preguntas para estudiar este proyecto

- Que clase recibe la solicitud inicial?
- Donde se aplica la regla de negocio?
- Que objeto representa la tabla, documento o dato principal?
- Que DTO evita exponer detalles internos?
- Que test confirma que el comportamiento se mantiene?

Microservicio de autenticacion

Tema principal: Spring Security, JWT, roles, login, validacion de token, Vault, Config Server y tests.

Analogia del proyecto: Es la porteria del edificio: identifica personas, revisa credenciales y decide que puertas puede abrir cada rol.

Dependencias y piezas visibles

Modulo Maven: spring-boot-starter-parent, junit, lombok, mockito, spring-boot-starter-data-jpa, spring-boot-starter-security, spring-boot-starter-test, spring-boot-starter-web, spring-cloud-starter-vault-config

Flujo mental

HTTP -> Controller -> Service -> Repository/Ciente externo -> Entity/DTO -> Response.

Clases y archivos explicados

Constants

Archivo: ms-auth/src/main/java/com/codigo/ms_auth/aggregates/constants/Constants.java

Tipo: class. Rol: DTO / request / response. Paquete: com.codigo.ms_auth.aggregates.constants.

- Transporta datos entre capas o hacia/desde HTTP sin exponer directamente la entidad.
- Sus datos/dependencias clave son: Boolean STATUS_ACTIVE, String CLAVE_AccountNonExpired, String CLAVE_AccountNonLocked, String CLAVE_CredentialsNonExpired, String CLAVE_Enabled, String REFRESH.
- Analogia: Como un formulario: solo lleva los datos necesarios para una operacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

SignInRequest

Archivo: ms-auth/src/main/java/com/codigo/ms_auth/aggregates/request/SignInRequest.java

Tipo: class. Rol: DTO / request / response. Paquete: com.codigo.ms_auth.aggregates.request.

- Transporta datos entre capas o hacia/desde HTTP sin exponer directamente la entidad.
- Sus datos/dependencias clave son: String email, String password.
- Metodos para revisar: SignInRequest(String email, String password).
- Anotaciones importantes: @Getter, @Setter.
- Analogia: Como un formulario: solo lleva los datos necesarios para una operacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

SignUpRequest

Archivo: ms-auth/src/main/java/com/codigo/ms_auth/aggregates/request/SignUpRequest.java

Tipo: class. Rol: DTO / request / response. Paquete: com.codigo.ms_auth.aggregates.request.

- Transporta datos entre capas o hacia/desde HTTP sin exponer directamente la entidad.
- Sus datos/dependencias clave son: String nombres, String apellido, String email, String password, String tipoDoc, String numDoc.
- Anotaciones importantes: @AllArgsConstructor, @Getter, @NoArgsConstructor, @Setter.
- Analogia: Como un formulario: solo lleva los datos necesarios para una operacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

SignInResponse

Archivo: ms-auth/src/main/java/com/codigo/ms_auth/aggregates/response/SignInResponse.java

Tipo: class. Rol: DTO / request / response. Paquete: com.codigo.ms_auth.aggregates.response.

- Transporta datos entre capas o hacia/desde HTTP sin exponer directamente la entidad.
- Sus datos/dependencias clave son: String token, String email, List<String> roles.
- Anotaciones importantes: @AllArgsConstructor, @Builder, @Getter, @NoArgsConstructor, @Setter.

- Analogia: Como un formulario: solo lleva los datos necesarios para una operacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

SignUpResponse

Archivo: ms-auth/src/main/java/com/codigo/ms_auth/aggregates/response/SignUpResponse.java

Tipo: class. Rol: DTO / request / response. Paquete: com.codigo.ms_auth.aggregates.response.

- Transporta datos entre capas o hacia/desde HTTP sin exponer directamente la entidad.
- Sus datos/dependencias clave son: String mensaje, String email.
- Anotaciones importantes: @AllArgsConstructor, @Getter, @Setter.
- Analogia: Como un formulario: solo lleva los datos necesarios para una operacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

JwtAuthenticationFilter

Archivo: ms-auth/src/main/java/com/codigo/ms_auth/config/JwtAuthenticationFilter.java

Tipo: class. Rol: Seguridad. Paquete: com.codigo.ms_auth.config.

- Configura autenticacion, autorizacion, JWT o filtros de acceso.
- Sus datos/dependencias clave son: final JwtService jwtService, final UsuarioService usuarioService.
- Metodos para revisar: void doFilterInternal(HttpServletRequest request, HttpServletResponse response, FilterChain filterChain).
- Anotaciones importantes: @Component, @Override, @RequiredArgsConstructor.
- Analogia: Como la porteria que revisa identidad y permisos.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

SecurityConfiguration

Archivo: ms-auth/src/main/java/com/codigo/ms_auth/config/SecurityConfiguration.java

Tipo: class. Rol: Seguridad. Paquete: com.codigo.ms_auth.config.

- Configura autenticacion, autorizacion, JWT o filtros de acceso.
- Sus datos/dependencias clave son: final JwtAuthenticationFilter jwtAuthenticationFilter, final UsuarioService usuarioService.
- Metodos para revisar: SecurityFilterChain securityFilterChain(HttpSecurity http), AuthenticationProvider authenticationProvider(), PasswordEncoder passwordEncoder(), AuthenticationManager authenticationManager(AuthenticationConfiguration config).
- Anotaciones importantes: @Bean, @Configuration, @EnableMethodSecurity, @EnableWebSecurity, @RequiredArgsConstructor.
- Analogia: Como la porteria que revisa identidad y permisos.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

AdminController

Archivo: ms-auth/src/main/java/com/codigo/ms_auth/controller/AdminController.java

Tipo: class. Rol: Controlador REST. Paquete: com.codigo.ms_auth.controller.

- Recibe solicitudes HTTP, toma parametros/body y delega al servicio.
- Expone rutas HTTP como @RequestMapping("/test"), @GetMapping("/superadmin"), @GetMapping("/admin"), @GetMapping("/user").
- Sus datos/dependencias clave son: final UsuarioRepository usuarioRepository.
- Metodos para revisar: ResponseEntity<?> getSuperAdmins(), ResponseEntity<?> getAdmins(), ResponseEntity<?> getUsers().
- Anotaciones importantes: @GetMapping, @PreAuthorize, @RequestMapping, @RequiredArgsConstructor, @RestController.
- Analogia: Como una ventanilla: escucha al usuario y pasa el pedido a la oficina correcta.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

AuthenticationController

Archivo: ms-auth/src/main/java/com/codigo/ms_auth/controller/AuthenticationController.java

Tipo: class. Rol: Controlador REST. Paquete: com.codigo.ms_auth.controller.

- Recibe solicitudes HTTP, toma parametros/body y delega al servicio.
- Expone rutas HTTP como @RequestMapping("/auth"), @PostMapping("/register"), @PostMapping("/login"), @GetMapping("/validate").
- Sus datos/dependencias clave son: final AuthenticationService authenticationService.
- Metodos para revisar: ResponseEntity<SignUpResponse> register(@RequestBody SignUpRequest signUpRequest), ResponseEntity<SignInResponse> login(@RequestBody SignInRequest signInRequest), ResponseEntity<?> validateToken(@RequestHeader("Authorization")).
- Anotaciones importantes: @GetMapping, @PostMapping, @RequestBody, @RequestHeader, @RequestMapping, @RequiredArgsConstructor, @RestController.
- Analogia: Como una ventanilla: escucha al usuario y pasa el pedido a la oficina correcta.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

UserController

Archivo: ms-auth/src/main/java/com/codigo/ms_auth/controller/UserController.java

Tipo: class. Rol: Controlador REST. Paquete: com.codigo.ms_auth.controller.

- Recibe solicitudes HTTP, toma parametros/body y delega al servicio.
- Expone rutas HTTP como @RequestMapping("/api/user/v1"), @GetMapping("/profile").
- Metodos para revisar: ResponseEntity<String> getUserProfile().
- Anotaciones importantes: @GetMapping, @RequestMapping, @RestController.
- Analogia: Como una ventanilla: escucha al usuario y pasa el pedido a la oficina correcta.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

Rol

Archivo: ms-auth/src/main/java/com/codigo/ms_auth/entity/Rol.java

Tipo: class. Rol: Entidad / modelo persistente. Paquete: com.codigo.ms_auth.entity.

- Representa datos de dominio y normalmente se mapea a tablas o documentos.
- Sus datos/dependencias clave son: String ADMIN, String USUARIO, String SUPERADMIN, Long id, String nombre.
- Metodos para revisar: String getAuthority().
- Anotaciones importantes: @AllArgsConstructor, @Builder, @Column, @Entity, @GeneratedValue, @Getter, @Id, @NoArgsConstructor.
- Analogia: Como una ficha oficial con campos que describen algo del negocio.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

Usuario

Archivo: ms-auth/src/main/java/com/codigo/ms_auth/entity/Usuario.java

Tipo: class. Rol: Entidad / modelo persistente. Paquete: com.codigo.ms_auth.entity.

- Representa datos de dominio y normalmente se mapea a tablas o documentos.
- Sus datos/dependencias clave son: Long id, String nombres, String apellidos, String email, String password, String tipoDoc.
- Metodos para revisar: Collection<? extends GrantedAuthority> getAuthorities(), String getUsername(), boolean isAccountNonExpired(), boolean isAccountNonLocked(), boolean isCredentialsNonExpired(), boolean isEnabled().
- Anotaciones importantes: @AllArgsConstructor, @Builder, @Entity, @GeneratedValue, @Getter, @Id, @JoinColumn, @JoinTable.
- Analogia: Como una ficha oficial con campos que describen algo del negocio.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

MsAuthApplication

Archivo: ms-auth/src/main/java/com/codigo/ms_auth/MsAuthApplication.java

Tipo: class. Rol: Punto de arranque. Paquete: com.codigo.ms_auth.

- Enciende Spring Boot, escanea componentes y levanta el servidor embebido.
- Metodos para revisar: void main(String[] args).
- Anotaciones importantes: @EnableDiscoveryClient, @SpringBootApplication.
- Analogia: Como girar la llave de contacto de un auto.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

RolRepository

Archivo: ms-auth/src/main/java/com/codigo/ms_auth/repository/RolRepository.java

Tipo: interface. Rol: Repositorio de datos. Paquete: com.codigo.ms_auth.repository.

- Abstrae la persistencia y permite consultar o guardar entidades.
- Analogia: Como el archivo o almacen donde se guarda informacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

UsuarioRepository

Archivo: ms-auth/src/main/java/com/codigo/ms_auth/repository/UsuarioRepository.java

Tipo: interface. Rol: Repositorio de datos. Paquete: com.codigo.ms_auth.repository.

- Abstrae la persistencia y permite consultar o guardar entidades.
- Analogia: Como el archivo o almacen donde se guarda informacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

AuthenticationService

Archivo: ms-auth/src/main/java/com/codigo/ms_auth/service/AuthenticationService.java

Tipo: interface. Rol: Contrato de servicio. Paquete: com.codigo.ms_auth.service.

- Define casos de uso disponibles sin revelar la implementacion.
- Analogia: Como el menu de un restaurante: dice que platos existen, no como se cocinan.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

AuthenticationServiceImpl

Archivo: ms-auth/src/main/java/com/codigo/ms_auth/service/impl/AuthenticationServiceImpl.java

Tipo: class. Rol: Servicio de negocio. Paquete: com.codigo.ms_auth.service.impl.

- Implementa reglas, validaciones, llamadas a repositorios o clientes externos.
- Sus datos/dependencias clave son: final UsuarioRepository usuarioRepository, final RolRepository rolRepository, final AuthenticationManager authenticationManager, final JwtService jwtService, final UsuarioService usuarioService.
- Metodos para revisar: SignUpResponse signUp(SignUpRequest signUpRequest), List<Usuario> todos(), boolean validateToken(String token), SignInResponse signIn(SignInRequest signInRequest), SignInResponse getTokenByRefreshToken(String token), ResponseEntity<?> validate(String authHeader), Usuario getUsuarioEntity(SignUpRequest signUpRequest).
- Anotaciones importantes: @Log4j2, @Override, @RequiredArgsConstructor, @Service.
- Analogia: Como la cocina: combina ingredientes y decide el proceso real.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

JwtServiceImpl

Archivo: ms-auth/src/main/java/com/codigo/ms_auth/service/impl/JwtServiceImpl.java

Tipo: class. Rol: Seguridad. Paquete: com.codigo.ms_auth.service.impl.

- Configura autenticacion, autorizacion, JWT o filtros de acceso.
- Sus datos/dependencias clave son: String secretKey.

- Metodos para revisar: Key getKey(), String extractUserName(String token), String generateToken(Usuario usuario), boolean validateToken(String token, UserDetails userDetails), String generateRefreshToken(Map<String, Object> extraClaims, UserDetails userDetails), boolean validateRefreshToken(String token), boolean isValidToken(String token).
- Anotaciones importantes: @Component, @Override, @Value.
- Analogia: Como la porteria que revisa identidad y permisos.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

UsuarioServiceImpl

Archivo: ms-auth/src/main/java/com/codigo/ms_auth/service/impl/UsuarioServiceImpl.java

Tipo: class. Rol: Servicio de negocio. Paquete: com.codigo.ms_auth.service.impl.

- Implementa reglas, validaciones, llamadas a repositorios o clientes externos.
- Sus datos/dependencias clave son: final UsuarioRepository usuarioRepository.
- Metodos para revisar: UserDetailsService userDetailsService(), UserDetails loadUserByUsername(String username).
- Anotaciones importantes: @Override, @RequiredArgsConstructor, @Service.
- Analogia: Como la cocina: combina ingredientes y decide el proceso real.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

JwtService

Archivo: ms-auth/src/main/java/com/codigo/ms_auth/service/JwtService.java

Tipo: interface. Rol: Seguridad. Paquete: com.codigo.ms_auth.service.

- Configura autenticacion, autorizacion, JWT o filtros de acceso.
- Analogia: Como la porteria que revisa identidad y permisos.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

UsuarioService

Archivo: ms-auth/src/main/java/com/codigo/ms_auth/service/UsuarioService.java

Tipo: interface. Rol: Contrato de servicio. Paquete: com.codigo.ms_auth.service.

- Define casos de uso disponibles sin revelar la implementacion.
- Analogia: Como el menu de un restaurante: dice que platos existen, no como se cocinan.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

AdminControllerTest

Archivo: ms-auth/src/test/java/com/codigo/ms_auth/controller/AdminControllerTest.java

Tipo: class. Rol: Prueba. Paquete: com.codigo.ms_auth.controller.

- Verifica comportamiento de una clase, endpoint o contexto.
- Sus datos/dependencias clave son: UsuarioRepository usuarioRepository, AdminController adminController.
- Anotaciones importantes: @BeforeEach, @InjectMocks, @Mock, @Test.
- Analogia: Como una lista de chequeo antes de despegar.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

AuthenticationControllerTest

Archivo: ms-auth/src/test/java/com/codigo/ms_auth/controller/AuthenticationControllerTest.java

Tipo: class. Rol: Prueba. Paquete: com.codigo.ms_auth.controller.

- Verifica comportamiento de una clase, endpoint o contexto.
- Sus datos/dependencias clave son: AuthenticationService authenticationService, AuthenticationController authenticationController.
- Metodos para revisar: void setUp(), void testRegister(), void testLogin(), void testValidateToken().
- Anotaciones importantes: @BeforeEach, @InjectMocks, @Mock, @Test, @email.
- Analogia: Como una lista de chequeo antes de despegar.

- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

UserControllerTest

Archivo: ms-auth/src/test/java/com/codigo/ms_auth/controller/UserControllerTest.java

Tipo: class. Rol: Prueba. Paquete: com.codigo.ms_auth.controller.

- Verifica comportamiento de una clase, endpoint o contexto.
- Anotaciones importantes: @Test.
- Analogia: Como una lista de chequeo antes de despegar.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

MsSeguridadApplicationTests

Archivo: ms-auth/src/test/java/com/codigo/ms_auth/MsSeguridadApplicationTests.java

Tipo: class. Rol: Prueba. Paquete: com.codigo.ms_auth.

- Verifica comportamiento de una clase, endpoint o contexto.
- Anotaciones importantes: @SpringBootTest, @Test.
- Analogia: Como una lista de chequeo antes de despegar.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

AuthServiceTest

Archivo: ms-auth/src/test/java/com/codigo/ms_auth/service/AuthServiceTest.java

Tipo: class. Rol: Prueba. Paquete: com.codigo.ms_auth.service.

- Verifica comportamiento de una clase, endpoint o contexto.
- Sus datos/dependencias clave son: UsuarioRepository usuarioRepository, JwtService jwtService, AuthenticationManager authenticationManager, AuthenticationServiceImpl authService.
- Metodos para revisar: void setUp(), void testSignIn().
- Anotaciones importantes: @BeforeEach, @InjectMocks, @Mock, @Test, @prueba.
- Analogia: Como una lista de chequeo antes de despegar.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

AuthenticationServiceImplTest

Archivo: ms-auth/src/test/java/com/codigo/ms_auth/service/impl/AuthenticationServiceImplTest.java

Tipo: class. Rol: Prueba. Paquete: com.codigo.ms_auth.service.impl.

- Verifica comportamiento de una clase, endpoint o contexto.
- Sus datos/dependencias clave son: AuthenticationServiceImpl authenticationService, UsuarioRepository usuarioRepository, RolRepository rolRepository, JwtService jwtService, UsuarioService usuarioService, AuthenticationManager authenticationManager.
- Metodos para revisar: Set<Rol> mockRoles(), Usuario usuarioConRol(String rolNombre).
- Anotaciones importantes: @BeforeEach, @InjectMocks, @Mock, @Test, @email, @mail.
- Analogia: Como una lista de chequeo antes de despegar.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

JwtServiceImplTest

Archivo: ms-auth/src/test/java/com/codigo/ms_auth/service/impl/JwtServiceImplTest.java

Tipo: class. Rol: Prueba. Paquete: com.codigo.ms_auth.service.impl.

- Verifica comportamiento de una clase, endpoint o contexto.
- Sus datos/dependencias clave son: JwtServiceImpl jwtService, Usuario usuario.
- Anotaciones importantes: @BeforeEach, @Test, @mail.
- Analogia: Como una lista de chequeo antes de despegar.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

Preguntas para estudiar este proyecto

- Que clase recibe la solicitud inicial?
- Donde se aplica la regla de negocio?
- Que objeto representa la tabla, documento o dato principal?
- Que DTO evita exponer detalles internos?
- Que test confirma que el comportamiento se mantiene?

Microservicio de productos

Tema principal: CRUD de productos, JPA, validacion contra ms-auth, configuracion centralizada y tests.

Analogia del proyecto: Es el almacen: registra productos, precios y stock, pero antes revisa que el visitante tenga permiso.

Dependencias y piezas visibles

Modulo Maven: spring-boot-starter-parent, junit, lombok, mockito, spring-boot-starter-data-jpa, spring-boot-starter-security, spring-boot-starter-test, spring-boot-starter-web, spring-cloud-starter-vault-config

Flujo mental

HTTP -> Controller -> Service -> Repository/Ciente externo -> Entity/DTO -> Response.

Clases y archivos explicados

Constants

Archivo: ms-productos/src/main/java/com/codigo/ms_productos/aggregates/constants/Constants.java

Tipo: class. Rol: DTO / request / response. Paquete: com.codigo.ms_productos.aggregates.constants.

- Transporta datos entre capas o hacia/desde HTTP sin exponer directamente la entidad.
- Sus datos/dependencias clave son: String URL_VALIDATE_TOKEN, String ROLE_ADMIN, String ROLE_SUPERADMIN.
- Analogia: Como un formulario: solo lleva los datos necesarios para una operacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

RestTemplateConfig

Archivo: ms-productos/src/main/java/com/codigo/ms_productos/config/RestTemplateConfig.java

Tipo: class. Rol: Configuracion. Paquete: com.codigo.ms_productos.config.

- Declara beans, filtros, clientes o ajustes para que Spring los administre.
- Metodos para revisar: RestTemplate restTemplate().
- Anotaciones importantes: @Bean, @Configuration.
- Analogia: Como el tablero de interruptores que conecta piezas del sistema.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

SecurityConfiguration

Archivo: ms-productos/src/main/java/com/codigo/ms_productos/config/SecurityConfiguration.java

Tipo: class. Rol: Seguridad. Paquete: com.codigo.ms_productos.config.

- Configura autenticacion, autorizacion, JWT o filtros de acceso.
- Metodos para revisar: SecurityFilterChain securityFilterChain(HttpSecurity http).
- Anotaciones importantes: @Bean, @Configuration, @EnableWebSecurity.
- Analogia: Como la porteria que revisa identidad y permisos.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

ProductoController

Archivo: ms-productos/src/main/java/com/codigo/ms_productos/controller/ProductoController.java

Tipo: class. Rol: Controlador REST. Paquete: com.codigo.ms_productos.controller.

- Recibe solicitudes HTTP, toma parametros/body y delega al servicio.
- Expone rutas HTTP como @RequestMapping("/productos"), @PostMapping, @GetMapping, @GetMapping("/{id}").
- Sus datos/dependencias clave son: final ProductoService productoService, final AuthValidator authValidator.
- Metodos para revisar: ResponseEntity<?> crear(@RequestBody Producto producto, @RequestHeader("Authorization"), ResponseEntity<?> listar(@RequestHeader("Authorization"), ResponseEntity<?> obtener(@PathVariable Long id, @RequestHeader("Authorization"), ResponseEntity<?> existe(@PathVariable Long id, @RequestHeader("Authorization"), ResponseEntity<?> actualizar(@PathVariable Long id, @RequestBody Producto producto, @RequestHeader("Authorization"),

ResponseEntity<?> eliminar(@PathVariable Long id, @RequestHeader("Authorization"), ProductoController(ProductoService productoService, AuthValidator authValidator)).

- anotaciones importantes: @DeleteMapping, @GetMapping, @PathVariable, @PostMapping, @PutMapping, @RequestBody, @RequestHeader, @RequestMapping.

- Analogía: Como una ventanilla: escucha al usuario y pasa el pedido a la oficina correcta.

- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

Producto

Archivo: ms-productos/src/main/java/com/codigo/ms_productos/entity/Producto.java

Tipo: class. Rol: Entidad / modelo persistente. Paquete: com.codigo.ms_productos.entity.

- Representa datos de dominio y normalmente se mapea a tablas o documentos.

- Sus datos/dependencias clave son: Long id, String nombre, Double precio, String categoria.

- anotaciones importantes: @AllArgsConstructor, @Builder, @Entity, @GeneratedValue, @Getter, @Id, @NoArgsConstructor, @Setter.

- Analogía: Como una ficha oficial con campos que describen algo del negocio.

- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

MsProductosApplication

Archivo: ms-productos/src/main/java/com/codigo/ms_productos/MsProductosApplication.java

Tipo: class. Rol: Punto de arranque. Paquete: com.codigo.ms_productos.

- Enciende Spring Boot, escanea componentes y levanta el servidor embebido.

- Métodos para revisar: void main(String[] args).

- anotaciones importantes: @EnableDiscoveryClient, @SpringBootApplication.

- Analogía: Como girar la llave de contacto de un auto.

- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

ProductoRepository

Archivo: ms-productos/src/main/java/com/codigo/ms_productos/repository/ProductoRepository.java

Tipo: interface. Rol: Repositorio de datos. Paquete: com.codigo.ms_productos.repository.

- Abstrae la persistencia y permite consultar o guardar entidades.

- anotaciones importantes: @Repository.

- Analogía: Como el archivo o almacén donde se guarda información.

- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

ProductoService

Archivo: ms-productos/src/main/java/com/codigo/ms_productos/service/ProductoService.java

Tipo: interface. Rol: Contrato de servicio. Paquete: com.codigo.ms_productos.service.

- Define casos de uso disponibles sin revelar la implementación.

- Analogía: Como el menú de un restaurante: dice que platos existen, no como se cocinan.

- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

AuthValidator

Archivo: ms-productos/src/main/java/com/codigo/ms_productos/service/util/AuthValidator.java

Tipo: class. Rol: Clase auxiliar. Paquete: com.codigo.ms_productos.service.util.

- Apoya una tarea específica dentro del proyecto.

- Sus datos/dependencias clave son: final RestTemplate restTemplate, final ObjectMapper objectMapper, String urlValidate.

- Métodos para revisar: boolean tokenValido(String token), boolean tieneAcceso(String token), JsonNode validar(String token), AuthValidator(RestTemplate restTemplate).

- anotaciones importantes: @Service, @Value.

- Analogia: Como una herramienta pequena en la caja de trabajo.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

ProductoServiceImpl

Archivo: ms-productos/src/main/java/com/codigo/ms_productos/serviceImpl/ProductoServiceImpl.java

Tipo: class. Rol: Servicio de negocio. Paquete: com.codigo.ms_productos.serviceImpl.

- Implementa reglas, validaciones, llamadas a repositorios o clientes externos.
- Sus datos/dependencias clave son: final ProductoRepository productoRepository.
- Metodos para revisar: Producto crearProducto(Producto producto), List<Producto> listarProductos(), Producto obtenerProducto(Long id), boolean existeProducto(Long id), Producto actualizarProducto(Long id, Producto producto), void eliminarProducto(Long id), ProductoServiceImpl(ProductoRepository productoRepository).
- Anotaciones importantes: @Override, @Service.
- Analogia: Como la cocina: combina ingredientes y decide el proceso real.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

ProductoControllerTest

Archivo: ms-productos/src/test/java/com/codigo/ms_productos/controller/ProductoControllerTest.java

Tipo: class. Rol: Prueba. Paquete: com.codigo.ms_productos.controller.

- Verifica comportamiento de una clase, endpoint o contexto.
- Sus datos/dependencias clave son: ProductoService productoService, AuthValidator authValidator, ProductoController productoController.
- Metodos para revisar: void setUp(), void testListarProductos(), void testCrearProducto(), void testActualizarProducto(), void testObtenerProducto(), void testExisteProductoConTokenValido(), void testListarProductosSinRolPermitidoRetornaForbidden().
- Anotaciones importantes: @BeforeEach, @InjectMocks, @Mock, @Test.
- Analogia: Como una lista de chequeo antes de despegar.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

MsSeguridadApplicationTests

Archivo: ms-productos/src/test/java/com/codigo/ms_productos/MsSeguridadApplicationTests.java

Tipo: class. Rol: Prueba. Paquete: com.codigo.ms_productos.

- Verifica comportamiento de una clase, endpoint o contexto.
- Anotaciones importantes: @SpringBootTest, @Test.
- Analogia: Como una lista de chequeo antes de despegar.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

AuthValidatorTest

Archivo: ms-productos/src/test/java/com/codigo/ms_productos/service/util/AuthValidatorTest.java

Tipo: class. Rol: Prueba. Paquete: com.codigo.ms_productos.service.util.

- Verifica comportamiento de una clase, endpoint o contexto.
- Sus datos/dependencias clave son: RestTemplate restTemplate, AuthValidator authValidator.
- Anotaciones importantes: @BeforeEach, @Test, @mail.
- Analogia: Como una lista de chequeo antes de despegar.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

ProductoServiceImplTest

Archivo: ms-productos/src/test/java/com/codigo/ms_productos/serviceImpl/ProductoServiceImplTest.java

Tipo: class. Rol: Prueba. Paquete: com.codigo.ms_productos.serviceImpl.

- Verifica comportamiento de una clase, endpoint o contexto.

- Sus datos/dependencias clave son: ProductoRepository productoRepository, ProductoServiceImpl productoService.
- Anotaciones importantes: @BeforeEach, @InjectMocks, @Mock, @Test.
- Analogia: Como una lista de chequeo antes de despegar.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

Preguntas para estudiar este proyecto

- Que clase recibe la solicitud inicial?
- Donde se aplica la regla de negocio?
- Que objeto representa la tabla, documento o dato principal?
- Que DTO evita exponer detalles internos?
- Que test confirma que el comportamiento se mantiene?

Microservicio de ordenes

Tema principal: Gestion de ordenes, integracion REST, validacion de usuario, JPA y tests.

Analogia del proyecto: Es la caja de pedidos: crea ordenes y consulta otros servicios antes de aceptar la operacion.

Dependencias y piezas visibles

Modulo Maven: spring-boot-starter-parent, junit, lombok, mockito, spring-boot-starter-data-jpa, spring-boot-starter-security, spring-boot-starter-test, spring-boot-starter-web, spring-cloud-starter-vault-config

Flujo mental

HTTP -> Controller -> Service -> Repository/Ciente externo -> Entity/DTO -> Response.

Clases y archivos explicados

Constants

Archivo: ms-ordenes/src/main/java/com/codigo/ms_ordenes/aggregates/constants/Constants.java

Tipo: class. Rol: DTO / request / response. Paquete: com.codigo.ms_ordenes.aggregates.constants.

- Transporta datos entre capas o hacia/desde HTTP sin exponer directamente la entidad.
- Sus datos/dependencias clave son: String URL_VALIDATE_TOKEN, String ROLE_USUARIO, String ROLE_ADMIN, String ROLE_SUPERADMIN, String URL_PRODUCTOS.
- Analogia: Como un formulario: solo lleva los datos necesarios para una operacion.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

RestTemplateConfig

Archivo: ms-ordenes/src/main/java/com/codigo/ms_ordenes/config/RestTemplateConfig.java

Tipo: class. Rol: Configuracion. Paquete: com.codigo.ms_ordenes.config.

- Declara beans, filtros, clientes o ajustes para que Spring los administre.
- Metodos para revisar: RestTemplate restTemplate().
- Anotaciones importantes: @Bean, @Configuration.
- Analogia: Como el tablero de interruptores que conecta piezas del sistema.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

SecurityConfiguration

Archivo: ms-ordenes/src/main/java/com/codigo/ms_ordenes/config/SecurityConfiguration.java

Tipo: class. Rol: Seguridad. Paquete: com.codigo.ms_ordenes.config.

- Configura autenticacion, autorizacion, JWT o filtros de acceso.
- Metodos para revisar: SecurityFilterChain securityFilterChain(HttpSecurity http).
- Anotaciones importantes: @Bean, @Configuration, @EnableWebSecurity.
- Analogia: Como la porteria que revisa identidad y permisos.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

OrdenController

Archivo: ms-ordenes/src/main/java/com/codigo/ms_ordenes/controller/OrdenController.java

Tipo: class. Rol: Controlador REST. Paquete: com.codigo.ms_ordenes.controller.

- Recibe solicitudes HTTP, toma parametros/body y delega al servicio.
- Expone rutas HTTP como @RequestMapping("/ordenes"), @PostMapping, @GetMapping.
- Sus datos/dependencias clave son: final OrdenService ordenService.
- Metodos para revisar: ResponseEntity<Orden> crearOrden(@RequestBody Orden orden, @RequestHeader("Authorization"), ResponseEntity<List<Orden>> listarOrdenes(@RequestHeader("Authorization")).

- Anotaciones importantes: @GetMapping, @PostMapping, @RequestBody, @RequestHeader, @RequestMapping, @RequiredArgsConstructor, @RestController.

- Analogia: Como una ventanilla: escucha al usuario y pasa el pedido a la oficina correcta.

- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

Orden

Archivo: ms-ordenes/src/main/java/com/codigo/ms_ordenes/entity/Orden.java

Tipo: class. Rol: Entidad / modelo persistente. Paquete: com.codigo.ms_ordenes.entity.

- Representa datos de dominio y normalmente se mapea a tablas o documentos.

- Sus datos/dependencias clave son: Long id, Long usuarioid, List<Long> productosIds, LocalDateTime fecha.

- Anotaciones importantes: @AllArgsConstructor, @Builder, @ElementCollection, @Entity, @GeneratedValue, @Getter, @Id, @NoArgsConstructor.

- Analogia: Como una ficha oficial con campos que describen algo del negocio.

- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

MsOrdenApplication

Archivo: ms-ordenes/src/main/java/com/codigo/ms_ordenes/MsOrdenApplication.java

Tipo: class. Rol: Punto de arranque. Paquete: com.codigo.ms_ordenes.

- Enciende Spring Boot, escanea componentes y levanta el servidor embebido.

- Metodos para revisar: void main(String[] args).

- Anotaciones importantes: @EnableDiscoveryClient, @SpringBootApplication.

- Analogia: Como girar la llave de contacto de un auto.

- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

OrdenRepository

Archivo: ms-ordenes/src/main/java/com/codigo/ms_ordenes/repository/OrdenRepository.java

Tipo: interface. Rol: Repositorio de datos. Paquete: com.codigo.ms_ordenes.repository.

- Abstrae la persistencia y permite consultar o guardar entidades.

- Anotaciones importantes: @Repository.

- Analogia: Como el archivo o almacen donde se guarda informacion.

- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

OrdenService

Archivo: ms-ordenes/src/main/java/com/codigo/ms_ordenes/service/OrdenService.java

Tipo: interface. Rol: Contrato de servicio. Paquete: com.codigo.ms_ordenes.service.

- Define casos de uso disponibles sin revelar la implementacion.

- Analogia: Como el menu de un restaurante: dice que platos existen, no como se cocinan.

- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

AuthValidator

Archivo: ms-ordenes/src/main/java/com/codigo/ms_ordenes/service/util/AuthValidator.java

Tipo: class. Rol: Clase auxiliar. Paquete: com.codigo.ms_ordenes.service.util.

- Apoya una tarea especifica dentro del proyecto.

- Sus datos/dependencias clave son: String urlValidate, final RestTemplate restTemplate.

- Metodos para revisar: UsuarioDTO validateToken(String token).

- Anotaciones importantes: @Component, @RequiredArgsConstructor, @Value.

- Analogia: Como una herramienta pequena en la caja de trabajo.

- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

UsuarioDTO

Archivo: ms-ordenes/src/main/java/com/codigo/ms_ordenes/service/util/UsuarioDTO.java

Tipo: class. Rol: Clase auxiliar. Paquete: com.codigo.ms_ordenes.service.util.

- Apoya una tarea especifica dentro del proyecto.
- Sus datos/dependencias clave son: Long id, List<String> roles, String email.
- Metodos para revisar: String getRol().
- Anotaciones importantes: @Getter, @Setter.
- Analogia: Como una herramienta pequena en la caja de trabajo.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

OrdenServiceImpl

Archivo: ms-ordenes/src/main/java/com/codigo/ms_ordenes/serviceImpl/OrdenServiceImpl.java

Tipo: class. Rol: Servicio de negocio. Paquete: com.codigo.ms_ordenes.serviceImpl.

- Implementa reglas, validaciones, llamadas a repositorios o clientes externos.
- Sus datos/dependencias clave son: final OrdenRepository ordenRepository, final AuthValidator authValidator, final RestTemplate restTemplate, String productosUrl.
- Metodos para revisar: Orden crearOrden(Orden orden, String token), List<Orden> listarOrdenes(String token), void validarProducto(Long idProd, String token).
- Anotaciones importantes: @Override, @RequiredArgsConstructor, @Service, @Value.
- Analogia: Como la cocina: combina ingredientes y decide el proceso real.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

OrdenControllerTest

Archivo: ms-ordenes/src/test/java/com/codigo/ms_ordenes/controller/OrdenControllerTest.java

Tipo: class. Rol: Prueba. Paquete: com.codigo.ms_ordenes.controller.

- Verifica comportamiento de una clase, endpoint o contexto.
- Sus datos/dependencias clave son: MockMvc mockMvc, OrdenService ordenService, ObjectMapper objectMapper.
- Anotaciones importantes: @AutoConfigureMockMvc, @Autowired, @Import, @MockBean, @Test, @WebMvcTest.
- Analogia: Como una lista de chequeo antes de despegar.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

MsOrdenesApplicationTests

Archivo: ms-ordenes/src/test/java/com/codigo/ms_ordenes/MsOrdenesApplicationTests.java

Tipo: class. Rol: Prueba. Paquete: com.codigo.ms_ordenes.

- Verifica comportamiento de una clase, endpoint o contexto.
- Anotaciones importantes: @SpringBootTest, @Test.
- Analogia: Como una lista de chequeo antes de despegar.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

AuthValidatorTest

Archivo: ms-ordenes/src/test/java/com/codigo/ms_ordenes/service/util/AuthValidatorTest.java

Tipo: class. Rol: Prueba. Paquete: com.codigo.ms_ordenes.service.util.

- Verifica comportamiento de una clase, endpoint o contexto.
- Sus datos/dependencias clave son: RestTemplate restTemplate, AuthValidator authValidator.
- Anotaciones importantes: @BeforeEach, @Test, @mail.
- Analogia: Como una lista de chequeo antes de despegar.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

OrdenServiceImplTest

Archivo: ms-ordenes/src/test/java/com/codigo/ms_ordenes/serviceImpl/OrdenServiceImplTest.java

Tipo: class. Rol: Prueba. Paquete: com.codigo.ms_ordenes.serviceImpl.

- Verifica comportamiento de una clase, endpoint o contexto.
- Sus datos/dependencias clave son: OrdenRepository ordenRepository, AuthValidator authValidator, RestTemplate restTemplate, OrdenServiceImpl ordenService.
- Metodos para revisar: OrdenServiceImplTest().
- Anotaciones importantes: @InjectMocks, @Mock, @Test.
- Analogia: Como una lista de chequeo antes de despegar.
- Como estudiarla: identifica primero que entra, que regla aplica y que devuelve o guarda.

Preguntas para estudiar este proyecto

- Que clase recibe la solicitud inicial?
- Donde se aplica la regla de negocio?
- Que objeto representa la tabla, documento o dato principal?
- Que DTO evita exponer detalles internos?
- Que test confirma que el comportamiento se mantiene?

Proyecto final ReactCodigo Tecsup

Tema principal: Frontend React con Vite, rutas, catalogo, carrito, pedidos, componentes, hook y servicio local.

Analogia del proyecto: React es la vitrina y el mostrador: organiza pantallas, botones, productos y experiencia de compra para el usuario.

Dependencias y piezas visibles

bootstrap, react, react-router-dom, tailwind, vite

Flujo mental

main.jsx -> rutas -> layout -> paginas -> componentes -> hook/servicio -> estado del navegador.

Clases y archivos explicados

eslint.config.js

Archivo: proyecto-final-react-codigo-tecsup/eslint.config.js

Rol: Clase auxiliar.

- Apoya una tarea especifica dentro del proyecto.
- Exporta: defineConfig.
- Depende de: js from @eslint/js, globals from globals, reactHooks from eslint-plugin-react-hooks, reactRefresh from eslint-plugin-react-refresh, { defineConfig, globalIgnores } from eslint/config.
- Analogia: Como una herramienta pequena en la caja de trabajo.
- Como estudiarlo: mira que estado maneja, que props recibe y que evento dispara.

useLocalStorage.js

Archivo: proyecto-final-react-codigo-tecsup/src/hooks/useLocalStorage.js

Rol: Hook React.

- Encapsula estado o comportamiento reutilizable.
- Exporta: function.
- Funciones/componentes: useLocalStorage.
- Hooks usados: useEffect, useLocalStorage, useState.
- Depende de: { useState, useEffect } from react.
- Analogia: Como una herramienta que varias pantallas pueden tomar prestada.
- Como estudiarlo: mira que estado maneja, que props recibe y que evento dispara.

productService.js

Archivo: proyecto-final-react-codigo-tecsup/src/services/productService.js

Rol: Servicio frontend.

- Aisla acceso a datos o funciones reutilizables fuera de la UI.
- Funciones/componentes: getProductos.
- Analogia: Como una pequena API local para el frontend.
- Como estudiarlo: mira que estado maneja, que props recibe y que evento dispara.

vite.config.js

Archivo: proyecto-final-react-codigo-tecsup/vite.config.js

Rol: Clase auxiliar.

- Apoya una tarea especifica dentro del proyecto.
- Exporta: defineConfig.
- Depende de: { defineConfig } from vite, react from @vitejs/plugin-react-swc, tailwindcss from @tailwindcss/vite.

- Analogia: Como una herramienta pequena en la caja de trabajo.
- Como estudiarlo: mira que estado maneja, que props recibe y que evento dispara.

App.jsx

Archivo: proyecto-final-react-codigo-tecsup/src/App.jsx

Rol: Componente React.

- Construye una pieza visual reutilizable o una pantalla.
- Exporta: function.
- Funciones/componentes: App.
- Depende de: AppRoutes from ./routes/AppRoutes.
- Analogia: Como una pieza de LEGO visual.
- Como estudiarlo: mira que estado maneja, que props recibe y que evento dispara.

Loader.jsx

Archivo: proyecto-final-react-codigo-tecsup/src/components/ui/Loader.jsx

Rol: Componente React.

- Construye una pieza visual reutilizable o una pantalla.
- Exporta: function.
- Funciones/componentes: Loader.
- Analogia: Como una pieza de LEGO visual.
- Como estudiarlo: mira que estado maneja, que props recibe y que evento dispara.

Navbar.jsx

Archivo: proyecto-final-react-codigo-tecsup/src/components/ui/Navbar.jsx

Rol: Componente React.

- Construye una pieza visual reutilizable o una pantalla.
- Exporta: function.
- Funciones/componentes: Navbar, actualizarContador, handleCarritoUpdated.
- Hooks usados: useEffect, useState.
- Depende de: { Link } from react-router-dom, { useState, useEffect } from react, logo from ../../assets/img/logo.jpg.
- Analogia: Como una pieza de LEGO visual.
- Como estudiarlo: mira que estado maneja, que props recibe y que evento dispara.

ProductCard.jsx

Archivo: proyecto-final-react-codigo-tecsup/src/components/ui/ProductCard.jsx

Rol: Componente React.

- Construye una pieza visual reutilizable o una pantalla.
- Exporta: function.
- Funciones/componentes: ProductCard.
- Depende de: React from react.
- Analogia: Como una pieza de LEGO visual.
- Como estudiarlo: mira que estado maneja, que props recibe y que evento dispara.

MainLayout.jsx

Archivo: proyecto-final-react-codigo-tecsup/src/layouts/MainLayout.jsx

Rol: Componente React.

- Construye una pieza visual reutilizable o una pantalla.
- Exporta: function.

- Funciones/componentes: MainLayout.
- Depende de: Navbar from ../components/ui/Navbar.
- Analogia: Como una pieza de LEGO visual.
- Como estudiarlo: mira que estado maneja, que props recibe y que evento dispara.

main.jsx

Archivo: proyecto-final-react-codigo-tecsup/src/main.jsx

Rol: Clase auxiliar.

- Apoya una tarea especifica dentro del proyecto.
- Depende de: React from react, ReactDOM from react-dom/client, App from ./App.jsx.
- Analogia: Como una herramienta pequena en la caja de trabajo.
- Como estudiarlo: mira que estado maneja, que props recibe y que evento dispara.

Carrito.jsx

Archivo: proyecto-final-react-codigo-tecsup/src/pages/Carrito.jsx

Rol: Componente React.

- Construye una pieza visual reutilizable o una pantalla.
- Exporta: function.
- Funciones/componentes: Carrito, eliminarProducto, vaciarCarrito, confirmarCompra.
- Hooks usados: useLocalStorage, useNavigate.
- Depende de: useLocalStorage from ../hooks/useLocalStorage, Swal from sweetalert2, { useNavigate } from react-router-dom.
- Analogia: Como una pieza de LEGO visual.
- Como estudiarlo: mira que estado maneja, que props recibe y que evento dispara.

Catalogo.jsx

Archivo: proyecto-final-react-codigo-tecsup/src/pages/Catalogo.jsx

Rol: Componente React.

- Construye una pieza visual reutilizable o una pantalla.
- Exporta: function.
- Funciones/componentes: Catalogo, agregarAlCarrito.
- Hooks usados: useEffect, useLocalStorage, useState.
- Depende de: { useEffect, useState } from react, ProductCard from ../components/ui/ProductCard, Loader from ../components/ui/Loader, useLocalStorage from ../hooks/useLocalStorage, productosData from ../assets/productos.json.
- Analogia: Como una pieza de LEGO visual.
- Como estudiarlo: mira que estado maneja, que props recibe y que evento dispara.

Home.jsx

Archivo: proyecto-final-react-codigo-tecsup/src/pages/Home.jsx

Rol: Componente React.

- Construye una pieza visual reutilizable o una pantalla.
- Exporta: function.
- Funciones/componentes: Home.
- Depende de: imageStore from ../assets/img/imageStore.jpg, logoStars from ../assets/img/logoStars.png, { Link } from react-router-dom.
- Analogia: Como una pieza de LEGO visual.
- Como estudiarlo: mira que estado maneja, que props recibe y que evento dispara.

Pedidos.jsx

Archivo: proyecto-final-react-codigo-tecsup/src/pages/Pedidos.jsx

Rol: Componente React.

- Construye una pieza visual reutilizable o una pantalla.
- Exporta: function.
- Funciones/componentes: Pedidos, guardarPedidos, cambiarEstado, toggleProductos.
- Hooks usados: useEffect, useState.
- Depende de: { useState, useEffect } from react.
- Analogia: Como una pieza de LEGO visual.
- Como estudiarlo: mira que estado maneja, que props recibe y que evento dispara.

AppRoutes.jsx

Archivo: proyecto-final-react-codigo-tecsup/src/routes/AppRoutes.jsx

Rol: Rutas React.

- Define navegacion entre paginas.
- Exporta: function.
- Funciones/componentes: AppRoutes.
- Depende de: { BrowserRouter as Router, Routes, Route } from react-router-dom, MainLayout from ../layouts/MainLayout, Home from ../pages/Home, Catalogo from ../pages/Catalogo, Carrito from ../pages/Carrito.
- Analogia: Como el mapa interno de una tienda.
- Como estudiarlo: mira que estado maneja, que props recibe y que evento dispara.

Preguntas para estudiar este proyecto

- Que clase recibe la solicitud inicial?
- Donde se aplica la regla de negocio?
- Que objeto representa la tabla, documento o dato principal?
- Que DTO evita exponer detalles internos?
- Que test confirma que el comportamiento se mantiene?

Cierre y siguientes pasos

Ya viste con fuerza: MVC, REST, JPA, relaciones, DTOs, servicios, repositorios, seguridad JWT, microservicios, Config Server, OpenFeign, consumo de APIs externas, testing y React.

Para completar el roadmap, estudia despues: OAuth2/OIDC, Actuator, Micrometer, Eureka, Gateway, Circuit Breaker con Resilience4j, @Transactional avanzado, bean scopes personalizados, AOP real y pruebas JPA con @DataJpaTest.